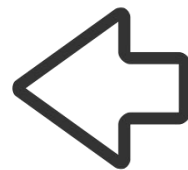


模块二：PyTorch与深度学习训练闭环

仿射拓扑、流形映射与 Autograd 动态图引擎逆向工程



主讲人：利群他爹



这是利群

callback

- 我们之前聊了logits回归的前向反向传播机制，但是如果在训练之前前向传播就已经把信号搞炸/搞没了怎么说，来看个例子：

- 任务：二分类，输入特征维度 $d = 100$ 。
- 模型： $z = \mathbf{w}^\top \mathbf{x} + b$, $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$ 。
- 损失：交叉熵 $\mathcal{L} = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$ 。
- 我们故意用 **极不合理的初始化**：权重 $w_i \sim \mathcal{N}(0, 10^2)$ ，偏置 $b = 0$ 。
- 取一个普通样本 $\mathbf{x}$ ，假设每个特征值都在 0.5 左右（比如来自 $\text{Uniform}(0, 1)$ ）。

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_{100} x_{100}$$

每个 w_i 的标准差是 10, $x_i \approx 0.5$ ，所以每一项 $w_i x_i$ 大致是标准差 $10 \times 0.5 = 5$ 的随机变量。

100 个这样的独立项相加，和的方差 = $100 \times 5^2 = 2500$ ，标准差 = 50。

也就是说，在还没训练的时候，仅仅因为随机初始化， z 的值就可能轻松跑到 ± 50 、 ± 100 这个量级。

例如某次采样得到 $z = 60$ 。

假设这个样本的真实标签 $y = 0$ ，但模型预测 $\hat{y} \approx 1$ 。

交叉熵损失：

$$\mathcal{L} = -0 \cdot \log(\hat{y}) - 1 \cdot \log(1 - \hat{y}) = -\log(1 - \hat{y}) \approx -\log(10^{-26}) = 26 \cdot \log(10) \approx 60$$

损失值巨大，这就是**信号爆炸**（loss 直接飙高）。

若 $y = 1$ 且 $\hat{y} \approx 1$ （即正确预测），梯度 $\partial \mathcal{L} / \partial z = 1 - 1 = 0$ ，权重完全得不到更新。更致命的是，对于深层网络，这种“饱和区”的局部梯度会连乘（Sigmoid 的最大导数才 0.25），导致前几层梯度彻底消失。

换言之

- 如果初始化方差更大 (如 $\text{Var}(w) = 100$) , z 的绝对值会超过数百, Sigmoid 的输出完全二值化 (0 或 1) , 而且计算 $\log(0)$ 会直接变成 NaN (Python 里 `-inf`) , 程序崩溃——这就是**信号爆炸**的极端表现。
- 如果初始化方差极小 (如 $\text{Var}(w) = 0.0001$) , 则 z 始终在 0 附近, Sigmoid 输出约 0.5, 模型输出毫无区分度, 梯度虽然不为零但非常小 ($0.5-0=0.5$) , 学习极度缓慢——这是**信号消失**。

那为了避免在训练开始之前, 模型直接爆炸或开始摆烂, 我们提出了一好玩意——流形体积守恒条件



天地有正气，杂然赋流形



- 这是文天祥的正气歌中一句诗，被北大的一个老登取来了，那什么是流形，如果你问高中的我，我会说：一个 n 维拓扑流形 M 是一个具有可数基 (countable base) 的拓扑 (topological) Hausdorff空间，该基局部同胚 (locally homeomorphic) 于 $\mathbb{R}^n$ 。这意味着对于 M 中的每一个点 p ，存在一个 p 的开邻域(open neighbourhood) U 和一个同胚(homeomorphism): $U \rightarrow \mathbb{R}^n$ ，它将集合 U 映射到一个开集合上。
- 养了利群之后，我会这么说：
- 利群趴在一张极大的地毯上，只需要 (x, y) 两个坐标就能确定位置
- 现在，我把这张地毯揉成一个极其复杂的团，扔进我们所在的三维 (3D) 空间里。
- 从外部观察者的上帝视角来看，这个团占据了 3D 空间，似乎需要 (x, y, z) 才能描述它。但这改变了纸张本身的维度吗？并没有。如果那只利群还在地毯上爬，它依然只需要感知二维坐标。

这就是流形：一个被高维空间 ‘嵌入 (Embed)’ 并扭曲的低维几何体。”

CV中的流形

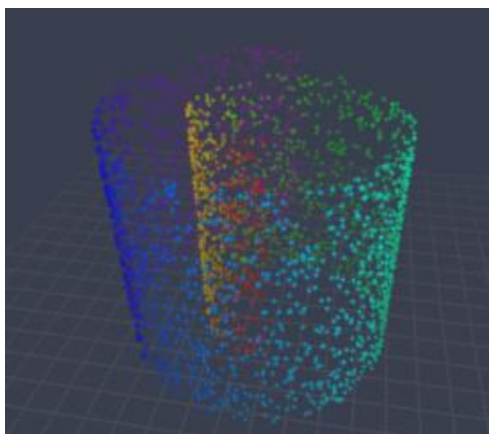
- 现在把视角拉回我们的老本行。一张 256×256 的 RGB 人脸图片，在计算机眼里是一个维度高达 196,608 维的向量。
- 如果这个 196,608 维的空间被均匀填满，那会是什么？那是无尽的彩色雪花噪点图。
- 但现实是，我们在这个超高维空间里随机抓取一个点，极大概率是一团雪花噪点，根本抓不到‘人脸’。
- 为什么？因为真正有意义的图像（如人脸、猫狗、铁轨）在这个 19 万维的空间里，只占据了一个极其狭小、极其薄的低维流形表面。比如，你把人脸向左转动 10 度，像素值发生了剧烈变化，但在流形上，它只是一条平滑、连续的低维轨迹。”



嘶，来点直观感受



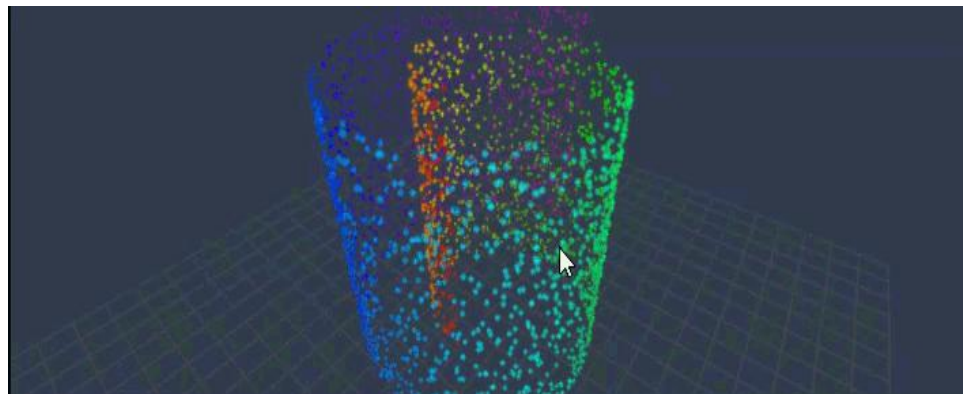
我有一张照片，他的shape是一个高维向量



他被存储在这个空间里

如果我们在这个被揉皱的‘照片纸团’上，一半画上好苹果（正样本），一半画上坏苹果（负样本）。在原始的高维空间里，这两种数据交织在一起，没有任何一条直线（线性分类器 $XW+b$ ）能把它们一刀切开。

那么，多层感知机（MLP）在干什么？每一次仿射变换（矩阵乘法）加上非线性激活（ReLU），就是一次对空间的拉伸、扭曲和折叠。深度神经网络的本质，就是一个极其强大的‘高维熨斗’。它一层层地把那个揉成一团的流形展开、铺平，直到最后在一个平坦的二维空间里，用一根简单的直线把正负样本切开。”



流形&梯度信号 (nn.Linear在干嘛)

1.1 矩阵乘法 = 旋转 + 拉伸/压缩

举个二维的例子，矩阵

$$W = \begin{pmatrix} 2 & 0 \\ 0 & 0.5 \end{pmatrix}$$

乘上向量 $\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$:

- x_1 被放大 2 倍 (拉伸)
- x_2 被缩小 0.5 倍 (压缩)

如果矩阵不是对角阵，还会发生**旋转** (就像把坐标系拧一下)。

所以 $\mathbf{XW}$ 就是在对输入空间做: **旋转 + 各向异性的拉伸/压缩**。

1.2 加上偏置 = 平移

$\mathbf{Z} = \mathbf{XW} + \mathbf{b}$, $\mathbf{b}$ 就是一个平移量。比如所有点都往右上方移动一点。

组合起来，就叫**仿射变换** (Affine Transformation)：

线性变换 (旋转、缩放) + 平移

1.1/1.2是不是就相当于我们把瑞士卷展开了

2.1 先理解“方差”在这里的意义

方差衡量数据的**分散程度**。如果一层输出全都是 0，方差就是 0，说明信号“死了”；如果输出数值动辄几百，方差巨大，说明信号“炸了”。

我们希望信息能平稳流过每一层：**输出方差和输入方差差不多**。

2.2 一个神经元的微观视角

假设我们只看一个输出神经元 z 。它由很多输入 $x_1, x_2, \dots, x_{d_{in}}$ 加权求和得到 (先忽略偏置，它不改变方差)：

$$z = x_1 w_1 + x_2 w_2 + \dots + x_{d_{in}} w_{d_{in}}$$

这里 w 是权重，初始时是随机数。假设：

- 每个输入 x_j 是随机变量，均值为 0，方差为 σ_x^2
- 每个权重 w_j 也是随机变量，均值 0，方差 σ_w^2
- 所有 x 和 w 彼此**独立** (没有互相影响)

next

2.3 计算 z 的方差

方差有一个重要性质（独立性下的方差加法）：

如果 A 和 B 独立，则 $\text{Var}(A + B) = \text{Var}(A) + \text{Var}(B)$ 。

如果 A 和 B 独立，则 $\text{Var}(A \times B) = \text{Var}(A)\text{Var}(B) + \text{Var}(A)E[B]^2 + \text{Var}(B)E[A]^2$ 。

当均值都为 0 时，简化为 $\text{Var}(A \times B) = \text{Var}(A)\text{Var}(B)$ 。

对每一项 $x_j w_j$ ：

- 均值为 0（因为 $E[x_j] = 0, E[w_j] = 0$ ）

- 方差 = $\text{Var}(x_j)\text{Var}(w_j) = \sigma_x^2 \sigma_w^2$

由于 z 是 d_{in} 个独立项之和：

$$\text{Var}(z) = \sigma_x^2 \sigma_w^2 + \cdots + \sigma_x^2 \sigma_w^2 \quad \text{共 } d_{in} \text{ 个}$$

$$\Rightarrow \text{Var}(z) = d_{in} \cdot \sigma_x^2 \cdot \sigma_w^2$$

写漂亮点：

$$\boxed{\text{Var}(z) \approx d_{in} \cdot \text{Var}(x) \cdot \text{Var}(w)}$$

2.4 关键的放大/缩小因子

注意 $d_{in} \cdot \text{Var}(w)$ 这个乘积：

- 如果它 > 1 ，每经过一层，方差就被放大一次。经过 L 层，方差变成原来的 $(d_{in} \text{Var}(w))^L$ 倍 → **指数爆炸**。
- 如果它 < 1 ，每层方差都缩小 → **指数塌缩**。

一个形象的数字：

假设 $d_{in} = 256$ ， $\text{Var}(w) = 0.01$ ，那么 $256 \times 0.01 = 2.56 > 1$ ，10 层后方差放大到 $2.56^{10} \approx 12000$ 倍！

如果 $\text{Var}(w) = 0.001$ ，因子是 0.256，10 层后变成 $0.256^{10} \approx 1.2 \times 10^{-6}$ ，信号几乎没了。

调控方式：前后向传播

深度学习训练的第一步，不是调学习率，而是先让信息能平稳流过网络。如果一开始信号就炸了或没了，后面根本学不起来。

所以我们希望每一层输入和输出的方差基本相等：

$$\text{Var}(z) \approx \text{Var}(x)$$

代入上面公式：

$$d_{in} \cdot \text{Var}(x) \cdot \text{Var}(w) \approx \text{Var}(x)$$

两边消去 $\text{Var}(x)$ （假设非零）：

$$\text{Var}(w) \approx \frac{1}{d_{in}}$$

这就是前向传播方差守恒条件。

为了让反向传播的梯度也能稳定流动（方差不变），我们希望反向“线性层”的输入输出方差相等，于是类似前向推导，得到：

$$\text{Var}(w) = \frac{1}{d_{out}}$$

因此，我们有两个都合情合理的要求：

- 前向看： $\text{Var}(w) = \frac{1}{d_{in}}$
- 反向看： $\text{Var}(w) = \frac{1}{d_{out}}$

在实际工程中 d_{in} 很难等于 d_{out} ，就有了Xavier 公式

最简单的折衷就是把两个条件的“目标方差”调和一下。一个自然的想法是：我们要一个方差值，既不要偏向 d_{in} ，也不要偏向 d_{out} ，那就取它们分母的算术平均再取倒数。即：

$$\text{Var}(w) = \frac{1}{\frac{d_{in} + d_{out}}{2}} = \frac{2}{d_{in} + d_{out}}$$

这就是 Glorot & Bengio (2010) 提出的 Xavier 初始化的核心公式。它不神奇，就是“前向要 $1/d_{in}$ ，反向要 $1/d_{out}$ ，那我取个中间值吧”的朴素思路。

理论给了我们需要的方差值，但实际初始化时我们是从一个均匀分布里随机抽值。均匀分布 $U[-a, a]$ 的方差是 $a^2/3$ 。我们令：

$$\frac{a^2}{3} = \frac{2}{d_{in} + d_{out}} \Rightarrow a = \sqrt{\frac{6}{d_{in} + d_{out}}}$$

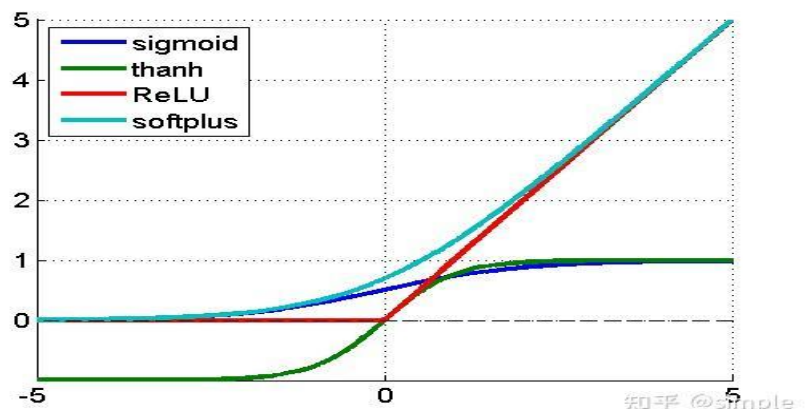
所以 Xavier 均匀分布 的区间是：

$$W \sim U\left[-\sqrt{\frac{6}{d_{in} + d_{out}}}, \sqrt{\frac{6}{d_{in} + d_{out}}}\right]$$

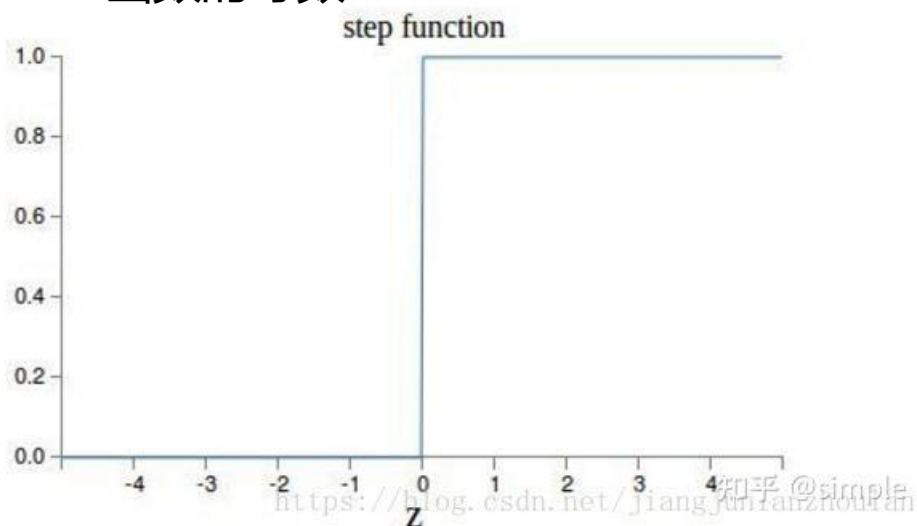
Kaiming (He) 初始化——为 ReLU 量身定制

- ReLU函数长这样：

$$f(x) = \max(0, x)$$



ReLU函数的导数



4.1 ReLU 如何吃掉一半方差

如果输入 z 是均值为 0 的对称分布，经过 ReLU 后，负的部分变成了 0，正的部分保持不变。输出 $h = \max(0, z)$ 的方差只有输入 z 方差的一半（严格推导需要计算 $E[h^2] = \frac{1}{2}E[z^2]$ 当分布对称时成立）。

4.2 补偿减半：方差翻倍

为了维持前向传播的方差不变，我们需要在初始化时给权重更多的“能量”。假如原本需要 $\text{Var}(w) = 1/d_{in}$ 来保持线性情况下的方差稳定，现在因为 ReLU 吃掉一半方差，我们必须让输出方差在过 ReLU 之前预放大 2 倍，这样才能抵消 ReLU 的减半效应。即：

$$\text{Var}(w) = \frac{2}{d_{in}}$$

这就是 Kaiming 初始化的核心。同样，它也有均匀分布和正态分布两种形式。

对于均匀分布 $U[-a, a]$ ，方差为 $a^2/3$ ，令其等于 $2/d_{in}$ ：

$$a = \sqrt{\frac{6}{d_{in}}}$$

若同样 $d_{in} = 100$ ，则 $a = \sqrt{0.06} \approx 0.245$ ，比 Xavier 的 0.2 稍大一点，正好弥补 ReLU 的损失。

对于正态分布，标准差 $\sqrt{2/d_{in}}$ 。

优化动力学，梯度的高效计算

- 初始化之后在此处键入公式。 ， 我们开始思考怎么高效计算梯度

一、全雅可比矩阵：一个你永远不想见到真身的庞然大物

假设我们的网络只有两个参数： w_1, w_2 。损失 $\mathcal{L}$ 是一个标量，那么梯度就是一个很友好的 2 维向量：

$$\nabla \mathcal{L} = \begin{pmatrix} \frac{\partial \mathcal{L}}{\partial w_1} \\ \frac{\partial \mathcal{L}}{\partial w_2} \end{pmatrix}$$

这很好。但是现在思考：如果我们要计算损失对中间某层输出的导数呢？比如中间层输出是一个 1000 维的向量 $\mathbf{h}$ 。那么 $\frac{\partial \mathcal{L}}{\partial \mathbf{h}}$ 就是一个 1000 维的向量，还是很友好。

问题出在“对参数求导”这一步。参数是矩阵 $\mathbf{W}$ ，形状 1000×500 ，包含 500,000 个独立标量。严格来说，损失对这些参数的全雅可比矩阵 $\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$ 是一个 $1 \times 500,000$ 的矩阵（或者写成一个 500,000 维的行向量）。虽然它仍然是向量，但当你有几十层、上千万参数时，它非常巨大。如果我们要显式存储所有层的这种行向量，数字直接爆炸。

但真正恐怖的是逐层雅可比连乘。考虑复合函数 $\mathcal{L} = f(g(h(\mathbf{W})))$ ，链式法则要求：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_3} \cdot \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} \cdot \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1} \cdot \frac{\partial \mathbf{h}_1}{\partial \mathbf{W}}$$

其中 $\frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1}$ 是一个雅可比矩阵，如果 $\mathbf{h}_1$ 和 $\mathbf{h}_2$ 都是 n 维向量，那么这个雅可比矩阵就是 $n \times n$ 的方阵。当 $n = 1000$ 时，这就是一个百万元素的矩阵！即使你能存下，计算它也是 $O(n^3)$ 的巨大开销。更可怕的是，深度学习里 n 可能达到上百万（比如大语言模型中间状态），全雅可比矩阵完全不可能显式存在。

于是我们面临一个窘境：理论上我们可以用链式法则一路推下去，但直接构造并连乘那些中间雅可比矩阵在计算上完全是自杀。

二、救命稻草：我们根本不需要完整的雅可比矩阵

仔细想，我们最终要的是什么？只是损失对每个参数的梯度，也就是一个长长的向量（梯度张量）。我们不需要那些中间雅可比矩阵本身，我们只需要它们乘以某个向量的结果。

比如，考虑一步反向传播：我们已经知道损失对某一层输出 $\mathbf{h}$ 的梯度（一个向量） $\mathbf{v} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}}$ 。现在想求损失对该层输入 $\mathbf{z}$ 的梯度 $\frac{\partial \mathcal{L}}{\partial \mathbf{z}}$ 。根据链式法则：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}} \cdot \frac{\partial \mathbf{h}}{\partial \mathbf{z}} = \mathbf{v}^T \mathbf{J}$$

其中 $\mathbf{J} = \frac{\partial \mathbf{h}}{\partial \mathbf{z}}$ 是那个庞大的雅可比矩阵。但是！我们不要 $\mathbf{J}$ 本身，我们只需要 $\mathbf{v}^T \mathbf{J}$ ，即向量 $\mathbf{v}$ 左乘矩阵 $\mathbf{J}$ 。

这就是 向量-雅可比积 (VJP)。神奇的是，我们可以不显式构造 $\mathbf{J}$ ，直接通过一个局部的反向函数计算出 $\mathbf{v}^T \mathbf{J}$ 。这就是每个 `autograd` 算子背后实现的东西。

用一个简单例子：假设 $\mathbf{h} = \mathbf{W}\mathbf{z}$ ，其中 $\mathbf{W}$ 是参数矩阵，输入 $\mathbf{z}$ 是向量。那么 $\frac{\partial \mathbf{h}}{\partial \mathbf{z}} = \mathbf{W}^T$ （雅可比矩阵就是权重矩阵的转置）。给定上游梯度 $\mathbf{v} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}}$ ，我们想得到 $\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \mathbf{v}^T \mathbf{W}^T = (\mathbf{W}\mathbf{v})^T$ ，也就是权重矩阵的转置乘以上游梯度向量。框架在实现 `linear` 的反向时，就只计算了 $\mathbf{W}^T \mathbf{v}$ ，而从未构造出那个巨大的 $\mathbf{W}^T$ 的副本。

所以，反向模式自动微分的本质就是：从输出端向输入端，不断计算 VJP，每个 VJP 只需要局部的信息（前向时缓存下来的值）和上游梯度向量，不需要任何大矩阵的显式构造。

没看懂？手推一下

一、雅可比矩阵是什么？先明确概念

对于一个函数 $y = f(\mathbf{x})$ ，输入 $\mathbf{x} \in \mathbb{R}^n$ ，输出 $\mathbf{y} \in \mathbb{R}^m$ 。它的雅可比矩阵 $\mathbf{J}$ 是一个 $m \times n$ 的矩阵，元素是：

$$J_{ij} = \frac{\partial y_i}{\partial x_j}$$

如果我们要计算损失 $\mathcal{L}$ 对 $\mathbf{x}$ 的梯度，链式法则给出：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \mathbf{J}$$

其中 $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ 是一个 $1 \times m$ 的行向量，乘以 $m \times n$ 的 $\mathbf{J}$ ，得到 $1 \times n$ 的行向量。

关键问题：如果 n 和 m 都很大，直接构造 $\mathbf{J}$ 并相乘是灾难。但很多时候 $\mathbf{J}$ 有特殊结构，使得这个乘法可以等价地用更便宜的操作完成。这就是 VJP 的实质。

手推一下：

考虑线性变换 $\mathbf{Y} = \mathbf{XW}$ ，其中 $\mathbf{X}$ 是 2×3 的矩阵， $\mathbf{W}$ 是 3×2 的矩阵，

$$\mathbf{Y} = \mathbf{XW}$$

- 输入 $\mathbf{X}$ 是一个 2×3 的矩阵 (batch size=2, 特征维度=3)
- 权重 $\mathbf{W}$ 是一个 3×2 的矩阵
- 输出 $\mathbf{Y}$ 是一个 2×2 的矩阵

为了简单，我们给出具体数值：

$$\mathbf{X} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}, \quad \mathbf{W} = \begin{pmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \\ 0.5 & 0.6 \end{pmatrix}$$

计算前向：

$$\mathbf{Y} = \mathbf{XW} = \begin{pmatrix} 1 \cdot 0.1 + 2 \cdot 0.3 + 3 \cdot 0.5 & 1 \cdot 0.2 + 2 \cdot 0.4 + 3 \cdot 0.6 \\ 4 \cdot 0.1 + 5 \cdot 0.3 + 6 \cdot 0.5 & 4 \cdot 0.2 + 5 \cdot 0.4 + 6 \cdot 0.6 \end{pmatrix} = \begin{pmatrix} 2.2 & 2.8 \\ 4.9 & 6.4 \end{pmatrix}$$

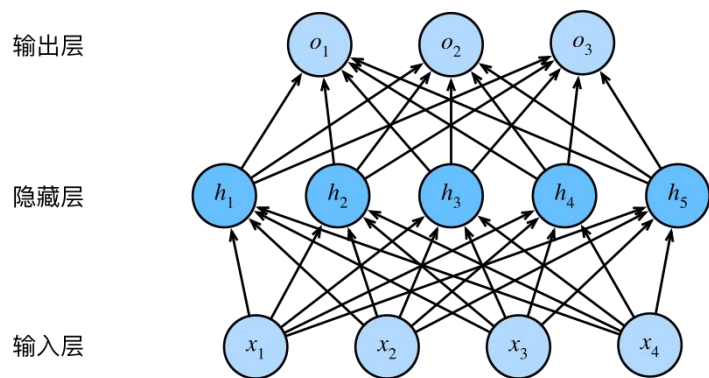
现在假设我们已得到损失对 $\mathbf{Y}$ 的梯度（上游梯度） $\frac{\partial \mathcal{L}}{\partial \mathbf{Y}}$ ，记为 $\mathbf{G}$ ，它和 $\mathbf{Y}$ 形状相同：

$$\mathbf{G} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

我们要计算对输入 $\mathbf{X}$ 的梯度 $\frac{\partial \mathcal{L}}{\partial \mathbf{X}}$ （形状 2×3 ）。

MLP模型，从线性到非线性

- 线性是个好东西，但生活中充满了非线性，猴桑交一个女朋友可能比开窍后谈一个吊一个，暧昧两个更加困难，这时候我们就很难用线性函数去描述了
- 我们可以通过在网络中加入一个或多个隐藏层来克服线性模型的限制，使其能处理更普遍的函数关系类型。要做到这一点，最简单的方法是将许多全连接层堆叠在一起。每一层都输出到上面的层，直到生成最后的输出。



推导

同之前的章节一样，我们通过矩阵 $\mathbf{X} \in \mathbb{R}^{n \times d}$ 来表示 n 个样本的小批量，其中每个样本具有 d 个输入特征。对于具有 h 个隐藏单元的单隐藏层多层感知机，用 $\mathbf{H} \in \mathbb{R}^{n \times h}$ 表示隐藏层的输出，称为隐藏表示（hidden representations）。在数学或代码中， $\mathbf{H}$ 也被称为隐藏层变量（hidden-layer variable）或隐藏变量（hidden variable）。因为隐藏层和输出层都是全连接的，所以我们有隐藏层权重 $\mathbf{W}^{(1)} \in \mathbb{R}^{d \times h}$ 和隐藏层偏置 $\mathbf{b}^{(1)} \in \mathbb{R}^{1 \times h}$ 以及输出层权重 $\mathbf{W}^{(2)} \in \mathbb{R}^{h \times q}$ 和输出层偏置 $\mathbf{b}^{(2)} \in \mathbb{R}^{1 \times q}$ 。形式上，我们按如下方式计算单隐藏层多层感知机的输出 $\mathbf{O} \in \mathbb{R}^{n \times q}$ ：

$$\begin{aligned}\mathbf{H} &= \mathbf{XW}^{(1)} + \mathbf{b}^{(1)}, \\ \mathbf{O} &= \mathbf{HW}^{(2)} + \mathbf{b}^{(2)}.\end{aligned}\tag{4.1.1}$$

注意在添加隐藏层之后，模型现在需要跟踪和更新额外的参数。可我们能从中得到什么好处呢？在上面定义的模型里，我们没有好处！原因很简单：上面的隐藏单元由输入的仿射函数给出，而输出（softmax操作前）只是隐藏单元的仿射函数。仿射函数的仿射函数本身就是仿射函数，但是我们之前的线性模型已经能够表示任何仿射函数。

我们可以证明这一等价性，即对于任意权重值，我们只需合并隐藏层，便可产生具有参数 $\mathbf{W} = \mathbf{W}^{(1)}\mathbf{W}^{(2)}$ 和 $\mathbf{b} = \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$ 的等价单层模型：

$$\mathbf{O} = (\mathbf{XW}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)} = \mathbf{XW}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)} = \mathbf{XW} + \mathbf{b}.\tag{4.1.2}$$

为了发挥多层架构的潜力，我们还需要一个额外的关键要素：在仿射变换之后对每个隐藏单元应用非线性的激活函数（activation function） σ 。激活函数的输出（例如， $\sigma(\cdot)$ ）被称为活性值（activations）。一般来说，有了激活函数，就不可能再将我们的多层感知机退化成线性模型：

$$\begin{aligned}\mathbf{H} &= \sigma(\mathbf{XW}^{(1)} + \mathbf{b}^{(1)}), \\ \mathbf{O} &= \mathbf{HW}^{(2)} + \mathbf{b}^{(2)}.\end{aligned}\tag{4.1.3}$$

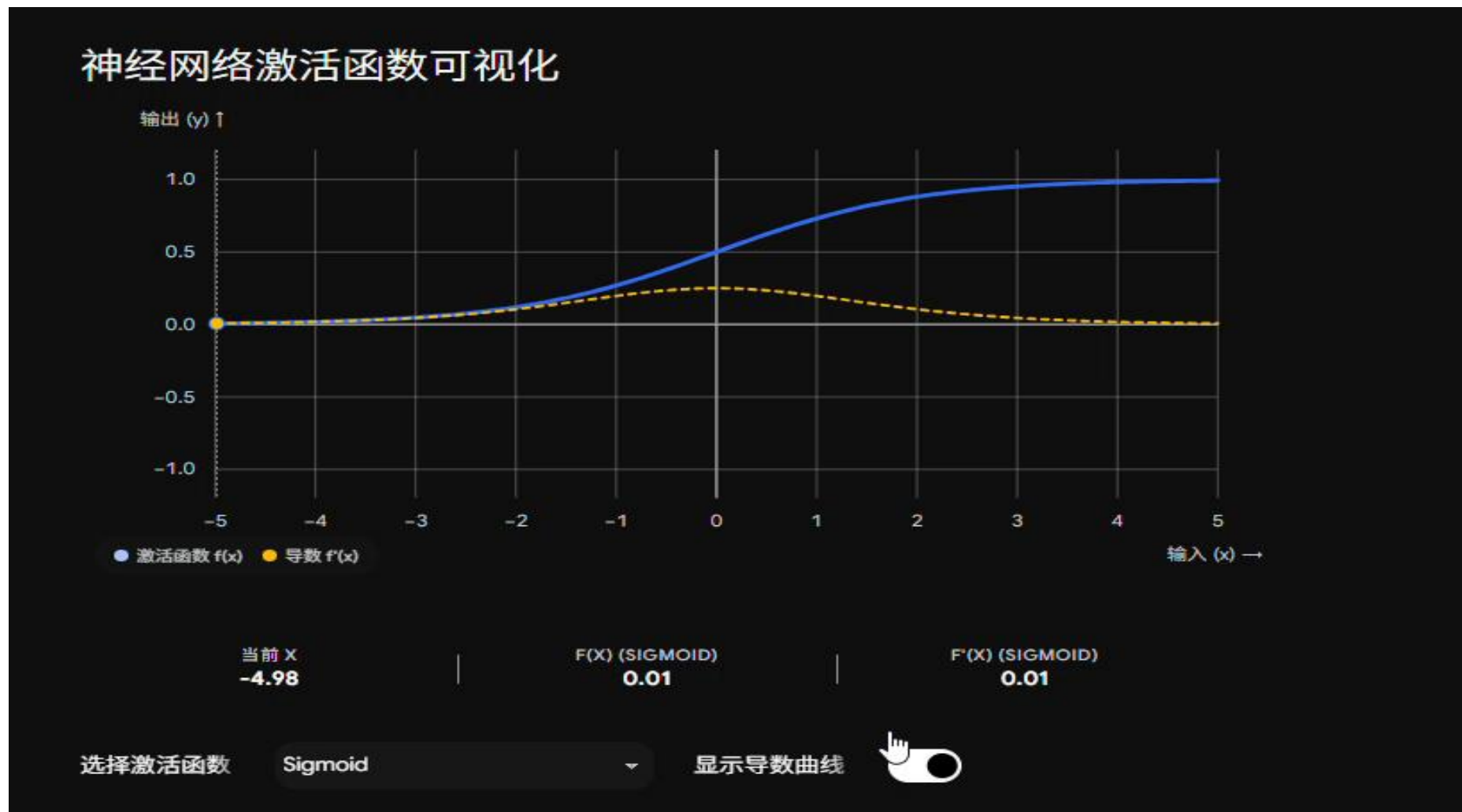
激活函数

深度学习核心激活函数架构级对比表

激活函数	数学表达式 $f(x)$	导数表达式 $f'(x)$	核心物理直觉与作用	架构师视角（缺陷与局限）
Sigmoid	$\frac{1}{1+e^{-x}}$	$f(x)(1 - f(x))$	将实数域压缩到 $(0, 1)$ ，提供概率解释。主要用于二分类的输出头，或控制信息流的“阀门”（如 LSTM 的门控机制）。	梯度衰减灾难： 导数最大值仅为 0.25 。非零中心化导致反向传播时，后层权重只能同向更新，引发极度低效的 Zig-zag（之字形）优化轨迹。
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	$1 - f(x)^2$	零中心化版本的 Sigmoid，输出域为 $(-1, 1)$ 。由于以 0 为中心，其梯度流比 Sigmoid 更健康，收敛速度更快。	严重饱和区： 输入绝对值 $ x > 3$ 时，雅可比矩阵迅速退化为零，深层网络中依然无法阻断梯度消失。
ReLU	$\max(0, x)$	$\mathbb{I}(x > 0)$ (分段常数)	深度学习的真正开端。在正半轴导数恒定为 1 ，完美抵抗了深层网络的链式连乘衰减；同时强迫网络产生高度稀疏的流形表征。	Dead ReLU (神经元脑死亡)： 若学习率过大引发权重剧变，使得输入被推入负半轴，其梯度将永久为 0 ，该神经元对应的拓扑维度彻底失效。
Leaky ReLU	$\max(\alpha x, x)$	$\begin{cases} 1, & x > 0 \\ \alpha, & x \leq 0 \end{cases}$	给予负半轴一个极小的梯度空间（如 $\alpha=0.01$ ），在维持非线性空间折叠能力的同时，为那些意外陷入负半轴的神经元提供一条微弱的“复活”通道。	引入了额外的超参数 α 。在超大规模工程验证中，其带来的涨点收益往往呈现边际递减。
GELU (Gaussian Error Linear Unit)	$x\Phi(x)$	$\Phi(x) + x\phi(x)$	现代大模型（ViT, LLMs）的绝对统治者。结合了 ReLU 的确定性与 Dropout 的随机正则化直觉；在 $x=0$ 零点附近极其平滑，不会造成梯度的突变震荡。	数学计算极其昂贵（涉及正态分布的累积分布函数 erf ），高度依赖底层 CUDA / TensorRT 的算子融合优化。



有意思的图像



作业

任务二：做梯度病理学实验

要求学生：

1. 故意使用极不合理的初始化，例如：
 - `normal_(std=1.0)`
2. 构造一个 10 层 MLP。
3. 在不同激活函数下：
 - ReLU
 - Sigmoid
 - GELU
4. 使用 Forward Hook 和 Backward Hook 记录每层：
 - 激活均值
 - 激活方差
 - 梯度均值
 - 梯度方差
 - 梯度谱范数或近似谱范数

要求输出数值证据，直接观察：

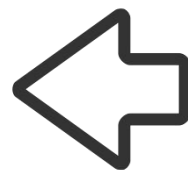
1. Sigmoid 网络前层梯度如何向极小量塌缩。
2. 不合理初始化如何放大大方差危机。

模块二：PyTorch与深度学习训练闭环

损失曲面的几何拓扑、动力系统与优化器底层推演



主讲人：利群他爹



这是利群

又是一次callback²，聊聊为什么loss会震荡

- 还记得我们之前讲过的二次求导的牛顿法吗
- 想象你站在一片连绵起伏的山脉里（这个山脉就是损失函数 $L(\theta)$ ），你的目标是走到最低的谷底。你现在的位置是 θ ，眼前的地形很复杂，你没法看清整片山脉的全貌。
- 但如果你只观察脚下很小的一圈，地面大致像一个二次曲面——有些方向是陡峭的山壁，有些方向是平缓的沟壑。这个“脚下的小圆圈”就是我们要做的二阶泰勒展开。

在一维，如果你有一个函数 $f(x)$ ，在 x_0 附近可以近似为：

$$f(x_0 + \delta) \approx f(x_0) + f'(x_0)\delta + \frac{1}{2}f''(x_0)\delta^2$$

- $f'(x_0)$ 是斜率，告诉你该往哪边走。
- $f''(x_0)$ 是曲率，告诉你地面是向上弯（正曲率）还是向下弯（负曲率），以及弯得有多厉害。

在多维情形，参数不是单个数字，而是一个向量 θ （比如神经网络的所有权重）。此时：

- 斜率变成了**梯度** $\mathbf{g} = \nabla \mathcal{L}(\theta)$ 。
- 曲率变成了**Hessian 矩阵** $\mathbf{H} = \nabla^2 \mathcal{L}(\theta)$ ，它记录了每一对方向之间的弯曲程度。

于是教案里的公式就出现了：

$$\mathcal{L}(\theta + \delta) \approx \mathcal{L}(\theta) + \mathbf{g}^\top \delta + \frac{1}{2}\delta^\top \mathbf{H} \delta$$

我们现在要研究的，就是这个局部二次函数的行为，因为它完全决定了梯度下降的更新规则。

梯度下降的稳定条件

假设损失函数在局部恰好是一个纯二次函数（没有一次项，因为我们在极小点附近展开时可以平移消除它）：

$$\mathcal{L}(\theta) = \frac{1}{2} \lambda \theta^2$$

这里 $\lambda > 0$ 是曲率（二阶导数）。它的梯度是：

$$g = \frac{d\mathcal{L}}{d\theta} = \lambda \theta$$

梯度下降的更新公式是：

$$\theta_{t+1} = \theta_t - \eta \cdot g_t = \theta_t - \eta \lambda \theta_t = (1 - \eta \lambda) \theta_t$$

你看，每走一步，参数就被乘以 $(1 - \eta \lambda)$ 。

- 如果 $|1 - \eta \lambda| < 1$ ，那么 $|\theta_{t+1}| < |\theta_t|$ ，参数越来越接近 0，训练收敛。
- 如果 $|1 - \eta \lambda| > 1$ ，参数会越来越大，离原点越来越远，训练发散。
- 如果 $1 - \eta \lambda$ 正好是负的，且绝对值小于 1，你会看到参数在正负之间交替，但绝对值逐渐减小（衰减震荡）。

所以收敛条件就是：

$$-1 < 1 - \eta \lambda < 1 \quad \Rightarrow \quad 0 < \eta < \frac{2}{\lambda}$$

学习率不能超过 $2/\lambda$ ，否则这一维度的参数就会炸掉。曲率越大（地面越陡），允许的学习率上限就越低。

$$f(x_0 + \delta) \approx f(x_0) + f'(x_0) \delta + \frac{1}{2} f''(x_0) \delta^2$$


hessian矩阵与分解

假设有一个 n 元实值函数 $f(x_1, x_2, \dots, x_n)$, 其Hessian矩阵 H 定义为: 

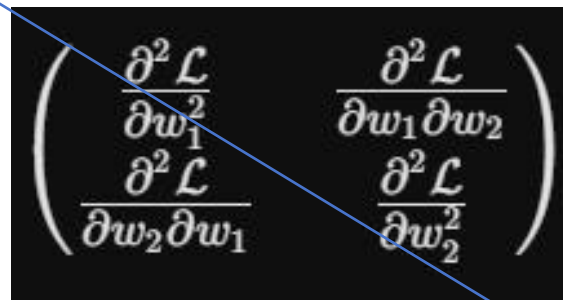
$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

hessian矩阵长这样, 实际上就是多元函数逐项求二阶偏导

我们一阶导数决定了函数的陡峭程度, 而二阶导数决定了函数的弯曲方向 (凹凸性)

$$\theta_{t+1} = (\mathbf{I} - \eta \mathbf{H}) \theta_t$$

我们的更新公式长这样


$$\begin{pmatrix} \frac{\partial^2 \mathcal{L}}{\partial w_1^2} & \frac{\partial^2 \mathcal{L}}{\partial w_1 \partial w_2} \\ \frac{\partial^2 \mathcal{L}}{\partial w_2 \partial w_1} & \frac{\partial^2 \mathcal{L}}{\partial w_2^2} \end{pmatrix}$$

我们随便举一个两参数的hessian, 我们会发现对角线上我们是好分析的

但是在非对角元素上一个诡异的事情发生了: 但就在 w_1 发生改变的那个极其微小的瞬间, w_2 那个方向的坡度 (梯度) 立刻被扭曲了!

在一个拥有 10亿参数的 Transformer 里, 如果 Hessian 矩阵是非对角化的, 意味着你更新第 1 层的某个权重, 可能会瞬间扭曲第 100层某个权重的曲率。百万维的参数互相拉扯, 就像一团乱麻, 如果用原始坐标系, 我们根本无法在数学上分析这个系统究竟是收敛还是爆炸。”

怎么整



手算：hessian的特征分解

- 先定义出两个参数：特征值和特征提示
- 特征值分解 ($H = Q\Lambda Q^T$) 通过改变基矢（旋转）来解决这个问题：特征向量（Q）：它们确定山谷的真实主轴。一个向量直接指向山谷底部；另一个向量直接指向陡峭的山谷壁。特征值（ Λ ）：它们给出沿这些新轴的纯粹的、非耦合的曲率（二阶导数）。

假设我们在一个两头高、中间低的椭圆中间。损失函数公式写出来是：

$$L = 5\theta_1^2 + 6\theta_1\theta_2 + 5\theta_2^2$$

- 注意中间那个 $6\theta_1\theta_2$ ：这就是交叉项（关联项）。
- 因为它，你改变 θ_1 的时候， θ_2 方向的坡度也跟着变了。
- 如果画出等高线，它是一个倾向了45度的椭圆形。

该函数的Hessian矩阵（求二阶导）是：

$$H = \begin{bmatrix} 10 & 6 \\ 6 & 10 \end{bmatrix}$$

令方程有非零解，矩阵 $H - \lambda I$ 的行列式必须为0，即 $\det(H - \lambda I) = 0$ 。

我们将 $H = \begin{bmatrix} 10 & 6 \\ 6 & 10 \end{bmatrix}$ 代入：

$$\det \begin{bmatrix} 10 - \lambda & 6 \\ 6 & 10 - \lambda \end{bmatrix} = 0$$

由此解得两个特征值：

- $\lambda_1 = 16$
- $\lambda_2 = 4$

根据二阶行列式计算公式($a_{11}a_{22} - a_{12}a_{21}$):

$$(10 - \lambda)(10 - \lambda) - 6 \times 6 = 0$$

我们注定要出的 λ 分别代回方程 $(H - \lambda I)q = 0$ 中，求出支持 $q = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ 。

1. 当 $\lambda_1 = 16$ 时间：

$$\begin{bmatrix} 10 - 16 & 6 \\ 6 & 10 - 16 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} -6 & 6 \\ 6 & -6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

这两个行化简后都对应一个方程： $-6x_1 + 6x_2 = 0 \implies x_1 = x_2$ 。

满足 $x_1 = x_2$ 我们先取最简单的一组基础解： $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ 。

$$Q = [q_1 \quad q_2] = \begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{2} \\ 1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix}$$

在三角函数中，我们知道 $\cos(45^\circ) = 1/\sqrt{2}$ ， $\sin(45^\circ) = 1/\sqrt{2}$ 。如果我们把它们代入，这个矩阵其实就是：

$$Q = \begin{bmatrix} \cos(45^\circ) & -\sin(45^\circ) \\ \sin(45^\circ) & \cos(45^\circ) \end{bmatrix}$$

这就是线性代数中经典的**逆时针旋转45°矩阵**！

而它的转置矩阵 Q^T ：

$$Q^T = \begin{bmatrix} \cos(45^\circ) & \sin(45^\circ) \\ -\sin(45^\circ) & \cos(45^\circ) \end{bmatrix} = \begin{bmatrix} \cos(-45^\circ) & -\sin(-45^\circ) \\ \sin(-45^\circ) & \cos(-45^\circ) \end{bmatrix}$$

2. 当 $\lambda_2 = 4$ 时间：

$$\begin{bmatrix} 10 - 4 & 6 \\ 6 & 10 - 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 6 & 6 \\ 6 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

化简后对应方程： $6x_1 + 6x_2 = 0 \implies x_1 = -x_2$ 。

取一组基础解： $\begin{bmatrix} -1 \\ 1 \end{bmatrix}$ 。

单位化：

长度同样是 $\sqrt{(-1)^2 + 1^2} = \sqrt{2}$ 。除以长度得到：

$$q_2 = \begin{bmatrix} -1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix}$$

令：

$$a = Q^T \theta$$

意思是：我们把原来的参数 θ （在标准坐标系下）投影到主方向 $q_1, q_2, \dots$ 上。新坐标 a_i 就是参数在主方向 q_i 上的分量。

反过来， $\theta = Qa$ （因为 Q 是正交矩阵， $Q^{-1} = Q^T$ ，所以转回去是乘以 Q ）。

现在把 $\theta = Qa$ 代入二次型：

$$\mathcal{L} = \frac{1}{2} \theta^T H \theta = \frac{1}{2} (Qa)^T H (Qa)$$

括号转置规则： $(Qa)^T = a^T Q^T$ ，所以：

$$\mathcal{L} = \frac{1}{2} a^T Q^T H Q a$$

但我们已经知道 $H = Q \Lambda Q^T$ ，所以：

$$Q^T H Q = Q^T (Q \Lambda Q^T) Q = (Q^T Q) \Lambda (Q^T Q)$$

由于 Q 是正交矩阵， $Q^T Q = I$ （单位矩阵），于是：

$$Q^T H Q = \Lambda$$

因此损失函数变为：

$$\mathcal{L} = \frac{1}{2} a^T \Lambda a$$

而 Λ 是对角阵，所以：

$$\mathcal{L} = \frac{1}{2} (\lambda_1 a_1^2 + \lambda_2 a_2^2 + \dots + \lambda_n a_n^2) = \frac{1}{2} \sum_{i=1}^n \lambda_i a_i^2$$

原来的梯度下降在 θ 空间是：

$$\theta_{t+1} = \theta_t - \eta H \theta_t$$

两边左乘 Q^T ：

$$Q^T \theta_{t+1} = Q^T \theta_t - \eta Q^T H \theta_t$$

因为 $a_t = Q^T \theta_t$ ， $a_{t+1} = Q^T \theta_{t+1}$ 。另外我们需要把右边的 $H \theta_t$ 也用 a_t 表示：

$$H \theta_t = H Q a_t$$

乘以 Q^T ：

$$Q^T H Q a_t = \Lambda a_t$$

所以更新方程变成：

$$a_{t+1} = a_t - \eta \Lambda a_t = (I - \eta \Lambda) a_t$$

因为 Λ 是对角阵，这个式子相当于对每个分量 $a^{(i)}$ 独立地：

$$a_{t+1}^{(i)} = (1 - \eta \lambda_i) a_t^{(i)}$$

完全和一维的公式一样！每一个“新坐标方向”都只根据自己的曲率 λ_i 衰减。

因为通过特征分解，我们发现：在歪空间里看起来错综复杂的梯度下降，本质上只是新空间里两个独立方向的直线运动。(a_1)方向（陡峭），每一步的步长都会被曲率16放大。如果学习率很大（比如大于(2/16)），(1 - 16\eta)就会变成负数，导致在这个方向上来回震荡甚至爆炸。在(a_2)方向（平缓），每一步的步长只被曲率4放大。为了在这个慢方向上走得快，我们希望(eta)大一点。这就是两难困境：容纳了陡峭方向 (eta)必须小)，平缓方向就走不动（震荡）；照顾了平缓方向 (eta)设大)，陡峭方向就爆炸。特征划分帮助我们彻底看清了：阻止梯度下降效率的，就是特征值的顶部与简单的比例（条件数）！

条件数：

现在我们有 n 个独立的一维更新。要让整个系统稳定，必须每个方向都稳定，也就是对所有 i 都要有：

$$|1 - \eta \lambda_i| < 1$$

这等价于：

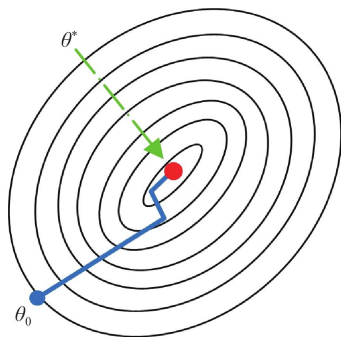
$$0 < \eta < \frac{2}{\lambda_{\max}}$$

其中 $\lambda_{\max}$ 是所有特征值里最大的那个。学习率的上限是被最陡峭的方向卡死的。如果你选的学习率稍微超过 $2/\lambda_{\max}$ ，那个最陡的方向就会立刻发散，其他方向再怎么稳定也没用。

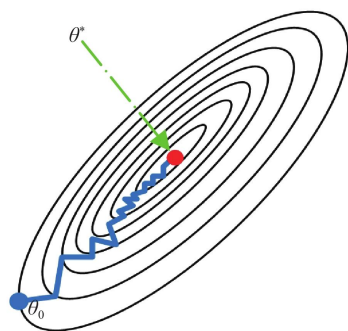
同时，最慢的收敛速度由最小的特征值 $\lambda_{\min}$ 决定。因为那个方向的乘子 $(1 - \eta \lambda_{\min})$ 最接近 1，意味着每一步收缩得最少。如果 $\lambda_{\min}$ 很小，那个方向就像蜗牛爬一样慢。

于是我们引出条件数：

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$$



(a) 较小的 κ
(a) Small κ



(b) 较大的 κ
(b) Large κ

- 如果 κ 接近 1，所有方向的曲率差不多，损失曲面像个球形碗，梯度下降可以大步流星地直奔谷底。
- 如果 κ 很大，就出现了病态曲率：有些方向极陡，有些方向极平。损失曲面变得像一条狭长的峡谷。

峡谷效应的直观解释：

- 陡峭的谷壁方向（大 λ_i ）逼迫你只能用很小的学习率，否则会震飞出去。
- 平缓的谷底方向（小 λ_i ）在小学习率下进展极其缓慢。
- 于是你看到 SGD 在陡壁之间来回剧烈震荡（因为乘子 $1 - \eta \lambda$ 接近 -1 或绝对值偏大），而沿着谷底却只能一点点挪动。

这就是教案里说的“之字形震荡”的数学根源：不是学习率调得不好，而是 Hessian 谱分布不均这个客观事实导致的必然行为。条件数越大，峡谷越险恶。

所以我们在用随机梯度下降法的时候，如果条件数很大，我们训练会很艰难，这和你的优化方式无关，是 SGD 自身的缺陷

就好比你们在理解这玩意的时候觉得很艰难，其实和你们太蠢无关（可能还是有点），而是这玩意机理本来就有点绕

牛爵爷的大宝贝——Momentum

- SGD 像一个没有质量的粒子，在一个极度粘稠的介质里移动。它的速度完全由当前位置的坡度决定（梯度），没有历史记忆。这就是一阶系统。
- Momentum 像一个有质量的球，在碗里滚动。它的运动由牛顿第二定律决定：加速度 = 力 / 质量。这里的“力”就是负梯度（指向碗底），此外还有摩擦力让它慢慢停下来。这就是二阶系统。

连续时间版本（牛顿第二定律 + 摩擦）：

$$m\ddot{\theta}(t) + c\dot{\theta}(t) + \nabla\mathcal{L}(\theta(t)) = 0$$

我来解释每一项：

- m ：质量。越大，惯性越强。
- $\ddot{\theta}(t)$ ：加速度，参数变化的“变化率”。
- c ：阻尼系数（摩擦力）。越大，能量耗散越快，球停得越快。
- $\dot{\theta}(t)$ ：速度，参数随时间的变化率。
- $\nabla\mathcal{L}$ ：势能场的负梯度，就是把球推向谷底的“恢复力”。

离散化：从微分方程到更新公式

计算机只能做离散步骤。我们把时间切成小段，每段长度对应学习率 η 。

用最简单的差分近似：

$$\ddot{\theta}(t) \approx \frac{\dot{\theta}_{t+1} - \dot{\theta}_t}{\eta}, \quad \dot{\theta}(t) \approx \dot{\theta}_t$$

代入方程，经过整理（这里只需要理解思路，具体推导可以之后自己补），会得到经典 Momentum 更新：

$$\mathbf{v}_{t+1} = \beta\mathbf{v}_t + (1 - \beta)\mathbf{g}_t$$

$$\theta_{t+1} = \theta_t - \eta\mathbf{v}_{t+1}$$

这里：

- $\mathbf{v}_t$ 是速度，它积累了历史梯度信息。
- $\beta \in [0, 1)$ 是动量系数，它和物理中的阻尼相关。 β 越接近 1，旧速度保留得越多，惯性越强。常用值如 0.9。
- $\mathbf{g}_t$ 是当前梯度。
- $(1 - \beta)$ 是新梯度的权重。



来看动量法的更新公式：

$$1. v_t = \beta v_{t-1} + (1 - \beta)g_{t-1}$$

$$2. v_{t-1} = \beta v_{t-2} + (1 - \beta)g_{t-2}$$

$$3. v_{t-2} = \beta v_{t-3} + (1 - \beta)g_{t-3}$$

$$\begin{aligned} v_t &= \beta v_{t-1} + (1 - \beta)g_t \\ &= \beta[\beta v_{t-2} + (1 - \beta)g_{t-1}] + (1 - \beta)g_t \\ &= \beta^2 v_{t-2} + \beta(1 - \beta)g_{t-1} + (1 - \beta)g_t \end{aligned}$$

我们不妨进行适当归纳：

把速度 $\mathbf{v}_{t+1}$ 的公式展开，可以看到它的本质：

$$\mathbf{v}_t = (1 - \beta) \sum_{k=0}^{t-1} \beta^k \mathbf{g}_{t-k}$$

- 这是对**所有历史梯度**做加权平均。
- 权重是 $(1 - \beta), (1 - \beta)\beta, (1 - \beta)\beta^2, \dots$ ，指数衰减。
- 越近的梯度权重越大，越远的越被遗忘。

这其实是一个**低通滤波器**。什么意思？

- 在陡峭方向上，梯度符号每步都在变（+g, -g, +g, -g...）。当对这些符号交替的值做加权平均时，正负项会互相抵消，平均后的值很小。所以震荡被“过滤”掉了。
- 在平缓方向上，梯度符号长期一致（一直为正或一直为负）。加权平均会把它们累加起来，速度持续增加。

这就是 Momentum 能在峡谷里加速的根本原因：它在不同方向上，根据梯度信号的“频率”，给予了不同的处理。高频震荡被压制，低频信号被保留并增强。

依旧hessian分析

我们手头有两维度的解耦一维方程（用 x 代替 a_i ， λ 代替 λ_i ）：

1. 速度更新： $v_{t+1} = \beta v_t + (1 - \beta)g_t$
2. 位置更新： $x_{t+1} = x_t - \eta v_{t+1}$
3. 在局部二次型下，当前点的梯度就是一阶导： $g_t = \lambda x_t$

现在的目标是：在公式里彻底消灭 v ，只留下 x 。

1. 从位置更新公式（公式2）里解出速度 v_{t+1} ：

$$\eta v_{t+1} = x_t - x_{t+1} \implies v_{t+1} = \frac{x_t - x_{t+1}}{\eta}$$

2. 同理，把时间往后退一步，写出 v_t 的表达式：

$$v_t = \frac{x_{t-1} - x_t}{\eta}$$

3. 把这两个速度表达式，以及 $g_t = \lambda x_t$ ，全部代入到速度更新公式（公式1）中：

$$\frac{x_t - x_{t+1}}{\eta} = \beta \left(\frac{x_{t-1} - x_t}{\eta} \right) + (1 - \beta) \lambda x_t$$

(注意：在这里，有的推导为了公式极简，会直接让 $g_t = \lambda x_t$ ，且不乘 $(1 - \beta)$ ，直接对应标准力学。这里我们直接按照你给出的经典标准形式强行凑项。)为了消去分母，我们两边同乘以 η ：

$$x_t - x_{t+1} = \beta(x_{t-1} - x_t) + \eta(1 - \beta)\lambda x_t$$

4. 展开并把 x_{t+1} 孤立到左边：

$$-x_{t+1} = -x_t + \beta x_{t-1} - \beta x_t + \eta(1 - \beta)\lambda x_t$$

两边同乘以 -1 ：

$$x_{t+1} = x_t - \beta x_{t-1} + \beta x_t - \eta(1 - \beta)\lambda x_t$$

5. 合并 x_t 的同类项：

$$x_{t+1} = (1 + \beta - \eta(1 - \beta)\lambda)x_t - \beta x_{t-1}$$

稳定条件变宽：从线性代数特征方程的根来分析（解二次方程的判别式），即使 $(1 + \beta - \eta\lambda)$ 对应的系数让当前项变大，前一步的 $(-\beta x_{t-1})$ 就像一根拉住刹车的弹簧（阻尼）。只要 (β) 选得合适，即使在极其陡峭的方向 (λ) 很大），原本在 SGD 里会直接飞出宇宙的学习率，在 Momentum 里也仅仅只会产生衰减的震荡，最终依然能收敛到谷底

多分类问题：CE

• callback一下我们的BCE：

我们来算一下 $\mathcal{L}_{BCE} = -\log p_t$ 对 z 的导数。

由链式法则：

$$\frac{\partial \mathcal{L}_{BCE}}{\partial z} = -\frac{1}{p_t} \cdot \frac{\partial p_t}{\partial z}$$

之前我们推导过：

$$\frac{\partial p_t}{\partial z} = y \cdot p_t(1 - p_t)$$

代入：

$$\frac{\partial \mathcal{L}_{BCE}}{\partial z} = -\frac{1}{p_t} \cdot y p_t(1 - p_t) = -y(1 - p_t)$$

这就是 BCE 的梯度，非常简洁。我们解读一下：

- 当样本被完美分类时, $p_t \rightarrow 1$, 梯度 $\rightarrow 0$, 不更新。
- 当样本完全分错时, $p_t \rightarrow 0$, 梯度绝对值 $\approx |-y| = 1$ (因为 $1 - p_t \approx 1$) , 模型得到一个强度为 1 的修正信号。
- 一般情况下, 梯度大小就是 $1 - p_t$, 正好等于“模型还差多少才完美”。

还记得让你们痛苦的鸢尾花吗，我们曾试图用一个二分类的方法去完成三分类，当然是有三分类适宜的直接方法。

就像让一帮学土木的学计算机专业都大概率听不太懂的课，当然这会更难，但不代表没有做下去的可能。whatever吧，你们新慈师姐的算法已经学到外太空去了，自己多想想差在哪。

来看softmax函数

对于向量 z 中的第 i 个元素，其 Softmax 值为：

$$S(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

$$L = \frac{1}{N} \sum_i L_i = -\frac{1}{N} \sum_i \sum_{c=1}^M y_{ic} \log(p_{ic})$$

其中：

- M ——类别的数量
- y_{ic} ——符号函数（0 或 1），如果样本 i 的真实类别等于 c 取 1，否则取 0
- p_{ic} ——观测样本 i 属于类别 c 的预测概率

先看softmax

$$p_i = \frac{e^{z_i}}{\sum_{k=1}^n e^{z_k}}$$

需要考虑的情况只有两种：当 $i = j$ （相同的索引）并且当 $i \neq j$ （不同的指标）。我们使用经

典的导数商法则： $\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$ 。

$$\begin{aligned}\frac{\partial p_i}{\partial z_i} &= \frac{(e^{z_i}) \cdot (\sum_{k=1}^n e^{z_k}) - (e^{z_i}) \cdot (e^{z_i})}{(\sum_{k=1}^n e^{z_k})^2} \\ &= \frac{e^{z_i}}{\sum_{k=1}^n e^{z_k}} \cdot \frac{\sum_{k=1}^n e^{z_k} - e^{z_i}}{\sum_{k=1}^n e^{z_k}} \\ &= p_i \cdot (1 - p_i)\end{aligned}$$

- 相同索引的结果： $\frac{\partial p_i}{\partial z_i} = p_i(1 - p_i)$

设分子为 $f = e^{z_i}$ 分母为 $g = \sum_{k=1}^n e^{z_k}$ 我们对.....求导 z_j ：

- 分子对.....的导数 z_j 是： $\frac{\partial f}{\partial z_j} = 0$ （因为 e^{z_i} 看待 z_j 作为常数）。
- 分母对.....的导数 z_j 是： $\frac{\partial g}{\partial z_j} = e^{z_j}$ 。

应用商法则：

$$\begin{aligned}\frac{\partial p_i}{\partial z_j} &= \frac{(0) \cdot (\sum_{k=1}^n e^{z_k}) - (e^{z_i}) \cdot (e^{z_j})}{(\sum_{k=1}^n e^{z_k})^2} \\ &= -\frac{e^{z_i}}{\sum_{k=1}^n e^{z_k}} \cdot \frac{e^{z_j}}{\sum_{k=1}^n e^{z_k}} \\ &= -p_i \cdot p_j\end{aligned}$$

- 不同指标的结果： $\frac{\partial p_i}{\partial z_j} = -p_i p_j$

还记得雅可比矩阵吗，相当于矩阵上的全微分

$$J = \frac{\partial \mathbf{p}}{\partial \mathbf{z}} = \begin{bmatrix} p_1(1 - p_1) & -p_1 p_2 & -p_1 p_3 \\ -p_2 p_1 & p_2(1 - p_2) & -p_2 p_3 \\ -p_3 p_1 & -p_3 p_2 & p_3(1 - p_3) \end{bmatrix}$$

CE的损失函数

$$J = \frac{\partial \mathbf{p}}{\partial \mathbf{z}} = \begin{bmatrix} p_1(1-p_1) & -p_1p_2 & -p_1p_3 \\ -p_2p_1 & p_2(1-p_2) & -p_2p_3 \\ -p_3p_1 & -p_3p_2 & p_3(1-p_3) \end{bmatrix}$$

对角线($i=j$)：全都是正数，代表各个类别自己分高了，自己的概率也变高。
非对角线($i \neq j$)：全部都是负数，代表其他类别分高了，自己的概率被挤压变低。

那我们再来想想这个损失函数怎么来的

$$\frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial p_1} \frac{\partial p_1}{\partial z_1} + \frac{\partial \mathcal{L}}{\partial p_2} \frac{\partial p_2}{\partial z_1} + \frac{\partial \mathcal{L}}{\partial p_3} \frac{\partial p_3}{\partial z_1}$$

对 $\mathcal{L} = -(y_1 \log p_1 + y_2 \log p_2 + y_3 \log p_3)$ 求导非常简单：

- $\frac{\partial \mathcal{L}}{\partial p_1} = -\frac{y_1}{p_1}$
- $\frac{\partial \mathcal{L}}{\partial p_2} = -\frac{y_2}{p_2}$
- $\frac{\partial \mathcal{L}}{\partial p_3} = -\frac{y_3}{p_3}$

回带雅克比矩阵

- 对于第一篇（同序， $i = j = 1$ ）： $\frac{\partial p_1}{\partial z_1} = p_1(1-p_1)$
- 对于第二项（不同编号， $i = 2, j = 1$ ）： $\frac{\partial p_2}{\partial z_1} = -p_2p_1$
- 对于第三项（不同编号， $i = 3, j = 1$ ）： $\frac{\partial p_3}{\partial z_1} = -p_3p_1$

把上面约分后的结果展开并整理：

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial z_1} &= -y_1 + y_1p_1 + y_2p_1 + y_3p_1 \\ &= -y_1 + p_1(y_1 + y_2 + y_3) \end{aligned}$$

因为 $\mathbf{y}$ 是One-hot标签（只有一个是1，其余是0），所以它们的和必然等于1（即 $y_1 + y_2 + y_3 = 1$ ）。

代入进去，公式直接缩写为：

$$\frac{\partial \mathcal{L}}{\partial z_1} = p_1 - y_1$$

同理，如果你对 z_2 求导，算出来的结果就是 $p_2 - y_2$ ；对 z_3 求导，就是 $p_3 - y_3$ 。
写成分块矩阵/支持形式，它就变成了：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}$$

CE 梯度的“优雅”

$$\mathcal{L} = -(y_1 \log p_1 + y_2 \log p_2 + y_3 \log p_3)$$

假设真实标签是 $y = [1, 0, 0]^T$ （即第一类是正确的），模型输出的未归一化分数（logits）是 $z = [z_1, z_2, z_3]^T$ 。

经过 Softmax 之后，预测概率为 $p_c = \frac{e^{z_c}}{\sum_k e^{z_k}}$ 。

此时的交叉熵损失函数为： $\mathcal{L}_{CE} = -\log p_1$ （因为只有 $y_1 = 1$ ，其余为0）。

我们来对正确的类别 z_1 求导（利用复合函数求导法则）：

$$\frac{\partial \mathcal{L}_{CE}}{\partial z_1} = -\frac{1}{p_1} \cdot \frac{\partial p_1}{\partial z_1}$$

根据 Softmax 的求导性质： $\frac{\partial p_1}{\partial z_1} = p_1(1 - p_1)$ 。代入上式：

$$\frac{\partial \mathcal{L}_{CE}}{\partial z_1} = -\frac{1}{p_1} \cdot p_1(1 - p_1) = -(1 - p_1) = p_1 - 1$$

我们再对错误的类别 z_2 求导：

$$\frac{\partial \mathcal{L}_{CE}}{\partial z_2} = -\frac{1}{p_1} \cdot \frac{\partial p_1}{\partial z_2}$$

根据 Softmax 跨项求导性质： $\frac{\partial p_1}{\partial z_2} = -p_1 p_2$ 。代入上式：

$$\frac{\partial \mathcal{L}_{CE}}{\partial z_2} = -\frac{1}{p_1} \cdot (-p_1 p_2) = p_2 = p_2 - 0$$

发现规律了吗？把它们拼回向量形式，正好是：

$$\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{z}} = \begin{bmatrix} p_1 - 1 \\ p_2 - 0 \\ p_3 - 0 \end{bmatrix} = \mathbf{p} - \mathbf{y}$$

- 教授为什么说它“优雅”？因为最终决定更新步长和方向的梯度，纯粹取决于预测概率和真实标签的差值。如果预测很准（ $p_1 \rightarrow 1$ ），梯度就接近 0；如果预测很差（ $p_1 \rightarrow 0$ ），梯度就接近 -1（全力反向修正）。

何爵爷的focal Loss

- $p_t = \sigma(yz)$ 。这里的 $y \in \{-1, +1\}$ 是一个数学技巧，一个式子。 σ 是 Sigmoid 函数。
- $\mathcal{L}_{FL} = -(1 - p_t)^\gamma \log p_t$ 。

步骤一：拆解 $\frac{\partial p_t}{\partial z} = y \cdot p_t(1 - p_t)$

这是对复合函数 $p_t = \sigma(yz)$ 求导。

令内层函数 $u = yz$ ，则 $\frac{\partial u}{\partial z} = y$ 。

外层函数是Sigmoid，我们知道 $\sigma(u)' = \sigma(u)(1 - \sigma(u))$ 。

根据链式法则：

$$\frac{\partial p_t}{\partial z} = \frac{\partial \sigma(u)}{\partial u} \cdot \frac{\partial u}{\partial z} = \sigma(u)(1 - \sigma(u)) \cdot y = y \cdot p_t(1 - p_t)$$

损失函数 $\mathcal{L}_{FL} = -(1 - p_t)^\gamma \cdot \log p_t$ 是两个关于 p_t 我们必须使用乘积求导法则 $(uv)' = u'v + uv'$ 。

为了澄清，我们先对 p_t 求导，最后再乘以 $\frac{\partial p_t}{\partial z}$ ：

- 令第一篇 $u = -(1 - p_t)^\gamma$ ，则 $\frac{\partial u}{\partial p_t} = \gamma(1 - p_t)^{\gamma-1}$
- 令第二项 $v = \log p_t$ ，则 $\frac{\partial v}{\partial p_t} = \frac{1}{p_t}$

应用乘积法则对 p_t 求导：

$$\frac{\partial \mathcal{L}_{FL}}{\partial p_t} = [\gamma(1 - p_t)^{\gamma-1}] \cdot \log p_t + [-(1 - p_t)^\gamma] \cdot \frac{1}{p_t}$$

为了凑出教案里的公因式 $(1 - p_t)^\gamma$ ，我们把第一个强行变形：因为 $(1 - p_t)^{\gamma-1} = \frac{(1-p_t)^\gamma}{1-p_t}$ ，所 现在，我们用链式法则乘以 $\frac{\partial p_t}{\partial z}$ （即步骤1算出的 $y \cdot p_t(1 - p_t)$ ）：
以：

$$\frac{\partial \mathcal{L}_{FL}}{\partial p_t} = \frac{\gamma(1 - p_t)^\gamma \log p_t}{1 - p_t} - \frac{(1 - p_t)^\gamma}{p_t}$$

通分，分母化为 $p_t(1 - p_t)$ ：

$$\frac{\partial \mathcal{L}_{FL}}{\partial p_t} = \frac{\gamma p_t(1 - p_t)^\gamma \log p_t - (1 - p_t)(1 - p_t)^\gamma}{p_t(1 - p_t)}$$

把分子上的 $(1 - p_t)^\gamma$ 提出，同时提取一个负号-1：

$$\frac{\partial \mathcal{L}_{FL}}{\partial p_t} = - \frac{(1 - p_t)^\gamma [(1 - p_t) - \gamma p_t \log p_t]}{p_t(1 - p_t)}$$

$$\begin{aligned} \frac{\partial \mathcal{L}_{FL}}{\partial z} &= \frac{\partial \mathcal{L}_{FL}}{\partial p_t} \cdot \frac{\partial p_t}{\partial z} \\ &= - \frac{(1 - p_t)^\gamma [(1 - p_t) - \gamma p_t \log p_t]}{p_t(1 - p_t)} \cdot (y \cdot p_t(1 - p_t)) \end{aligned}$$

看！分母的 $p_t(1 - p_t)$ 被完美约掉了！

剩下的一项整理一下（把y前面提到的最前面，中间里的减号和前面的负号做的）：

$$\frac{\partial \mathcal{L}_{FL}}{\partial z} = -y(1 - p_t)^\gamma [(1 - p_t) + \gamma p_t \log p_t]$$

何爵爷的千层套路

$$\frac{\partial \mathcal{L}_{FL}}{\partial z} = -y(1 - p_t)^\gamma [(1 - p_t) + \gamma p_t \log p_t]$$

1. 层次一： $\gamma = 0$ 时蜕为CE

如果令 $\gamma = 0$ ：

- 前面的阻尼器 $(1 - p_t)^0 = 1$
- 硬度里面的校正项 $\gamma p_t \log p_t = 0$
- 公式剩余： $\frac{\partial \mathcal{L}_{FL}}{\partial z} = -y(1 - p_t)$

说明 Focal Loss 是交叉熵在更高维度上的泛化，逻辑自治。

2. 层次二：前面的 $(1 - p_t)^\gamma$ 是“显式阻尼器”

在长尾数据中，简单样本（Easy Samples，比如背景、大多数类）非常多。它们的预测概率 p_t 非常接近1（比如 $p_t = 0.99$ ）。

- 此时， $(1 - p_t) = 0.01$ 。
- 如果 $\gamma = 2$ ，那么 $(1 - p_t)^2 = 0.0001$ 。
- 这一项作为乘法系数，直接把整个网络的总梯度压缩了一万倍！这意味着数量巨大的简单样本虽然多，但每个样本贡献的梯度接近于0，整个网络的总梯度不会被它们霸占。

3. 层次三：后面的 $\gamma p_t \log p_t$ 是“校正项”

这是普通人看不到的深度。很多人以为焦点损失只是损失函数外侧乘了一个权重系数 α 但如果是乘一个普通系数，根据求导法则，细微的内部结构是不变的。

但是，因为公式中出现了 $\gamma p_t \log p_t$ ，它改变了梯度随 p_t 变化的动态轨迹（局部减弱结构）。

- 当样本很困难时（ $p_t \rightarrow 0$ ）， $\log p_t$ 会变成一个很大的负数，这反过来会对抗前面的压缩，主动放大故障样本的最小最小值。
- 这就是所谓的“梯度手术”：它不是粗暴地把标签0改成0.1（比如标签平滑标签平滑），而是像手术刀一样，重新纵向了反向传播时，投影信号在通道里的缩放规则。

关于 γ ，我失去了一篇论文

SGD的BUG

在标准梯度下降法（SGD）中，需要防止过拟合。可以通过两种不同的方式来实现：

1. **L2 正则化**：在损失函数中添加惩罚项： $\mathcal{L}_{reg} = \mathcal{L} + \frac{\lambda}{2} \|\theta\|_2^2$ 当你求导时，梯度变为 $\mathbf{g}_t + \lambda\theta_t$ 。
2. **权重衰减**：不要改变损失函数。而是强制将权重按很小的百分比减小 $(1 - \eta\lambda)$ 就在添加渐变之前。

让我们来看一下使用 L2 正则化时的 SGD 方程：

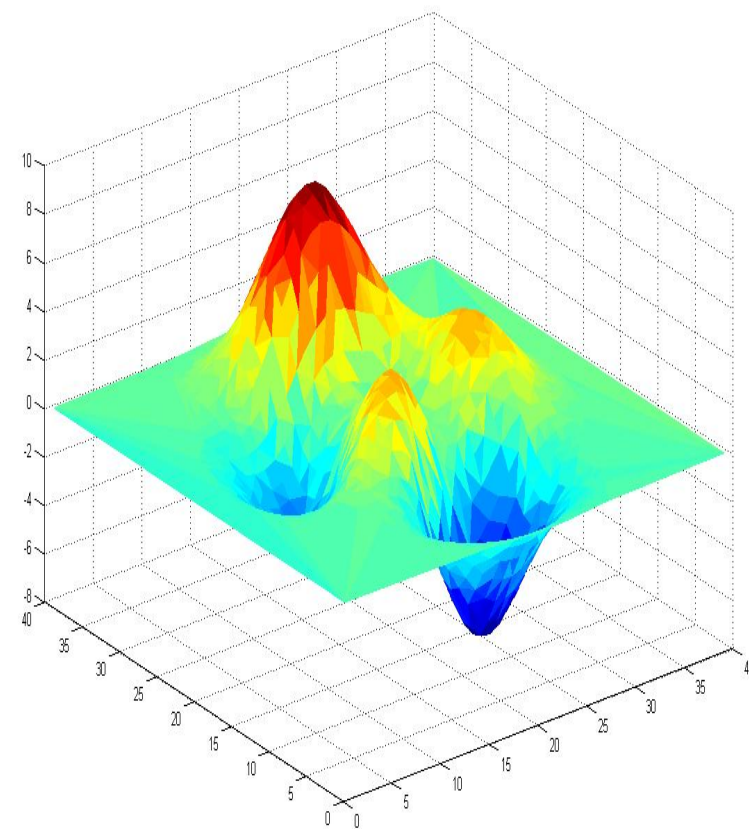
$$\theta_{t+1} = \theta_t - \eta(\mathbf{g}_t + \lambda\theta_t)$$

分配学习率 η ：

$$\theta_{t+1} = \theta_t - \eta\mathbf{g}_t - \eta\lambda\theta_t$$

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\mathbf{g}_t$$

瞧瞧： $(1 - \eta\lambda)\theta_t$ 这正是权重衰减！它以恒定的均匀速率将权重向零递减。因此，在随机梯度下降（SGD）中，L2 正则化和权重衰减在数学上是等价的。



妈的这几天太忙了，讲到这里算了

ADAM算法

1. 核心理念：一台机器里装两个引擎

Adam本质上是两种先前优化技术的混合体：动量法和RMSprop法。它跟踪网络中每个参数的两个不同的历史指标：

1. 第一时刻 (m_t) —— 《速度》(出自《动量》)：

1. 它追踪梯度的平均方向。
2. 如果某个参数持续获得正梯度，Adam 算法会朝着该方向加速。这有助于平滑噪声，并使模型越过平坦区域。

2. 第二时刻 (v_t — “非中心方差”(来自 RMSprop))：

1. 它跟踪梯度的原始幅度 (平方值)，忽略正负号。
2. 这可以作为衡量特定参数波动性或活跃程度的指标。

动量法已经提过了，再补充一下
RMSprop法



杰弗里·辛顿 (Geoffrey Hinton)，依旧计算机二分类，发际线感人

RMSprop

- 这时，最基础的陡峭下降（SGD）遇到了一个巨大的痛点：如果不同方向的陡峭大小悬殊不同，模型会在某个方向上眼神震荡，在另一个方向上进展缓慢。为了解决这个“震荡”问题，杰弗里·辛顿（Geoffrey Hinton）教授在2012年的Coursera课程中首次提出了RMSprop（Root Mean Square Propagation，均方根传播）。有趣的是，它甚至没有在正式论文中发表，而是直接在PPT里改变了深度学习界。
- RMSprop 的核心诉求是：如果某些参数的渐变特别大（震荡变焦），就强制缩小它的步长；如果某些参数的渐变特别小（进展缓慢），就放大它的步长。它通过追踪每个参数的最小程度到最小程度来达到该参数的“活跃度”或“震荡度”。

数学原理

第一步：计算梯度的指数移动平均平方（平滑的波动程度）

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2$$

- g_t 是当前步的读数。
- g_t^2 是梯度的平方（无论是正梯度还是负梯度，平方后都是正数，代表了绝对的视觉程度）。
- β 是衰减率（通常设为0.9），意味着它会更看重最近的瞬时大小，而逐渐忘记掉很久之前的瞬时大小。
- v_t 就是均方（Mean Square）的体现。

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \cdot g_t$$

- η 是初始学习率。
- ϵ 是一个极小数（如 10^{-8} ），防止分母为0。
- $\sqrt{v_t}$ 就是根（Root）的体现。

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1 - \beta) \mathbf{g}_t$$

$$\theta_{t+1} = \theta_t - \eta \mathbf{v}_{t+1}$$

以上是动量法的更新，主要看差异

$$\theta_{t+1} = \theta_t - \eta g_t$$

这是SGD的更新公式

缝合怪Adam

1. 独立运行的两套流水线

对每一个参数，Adam 同时计算两套指数移动平均（EMA）：

- **动量流水线（分子）**：不带平方，保留方向。

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

这里的 m_t 继承了动量法，用来决定**往哪里走**（惯性和方向）。

- **RMSprop流水线（分母）**：带平方，抹去方向。

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

这里的 v_t 继承了 RMSprop，用来决定**每一步走多大**（自适应步长）。

2. 合二为一的瞬间

在完成了各自的偏差修正（Bias Correction）后，Adam 开启了它的合体公式：

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$

- **对比标准 RMSprop**：标准 RMSprop 的分子是当前步的实时梯度 g_t 。Adam 把分子替换成了具有惯性的平滑方向 $\hat{m}_t$ 。
- **对比标准动量法**：标准动量法的分母是固定的 1。Adam 在分母位置安插了 RMSprop 的波动调节器 $\sqrt{\hat{v}_t}$ 。

还记得何爵爷的focallaoss吗

1. 核心异同对比

- **RMSprop 的思路**：从“梯度优化层”下手（物理加速）

- **机制**：它不管你的数据属于什么类别，它只看**参数的更新频率**。罕见特征对应的参数长期没有梯度，它的分母 $\sqrt{v_t}$ 就会变得极小。一旦该特征出现，哪怕梯度微弱，乘以 $\frac{1}{\sqrt{v_t}}$ 后也会被**瞬间放大**。
- **特点**：它是**参数级（Parameter-wise）**的自适应。

- **Focal Loss 的思路**：从“损失函数层”下手（降权简单样本）

- **机制**：它通过修改 Loss 公式，引入调节因子 $(1 - p_t)^{\gamma}$ 。对于那些模型已经预测得很准的“简单样本”（通常是长尾中的头部常见类），Loss 会被**暴风式压缩**；而对于预测不准的“困难/罕见样本”，Loss 保持原样。
- **特点**：它是**样本级（Sample-wise）**的自适应。

2. 实现方式的根本区别（以一个稀疏特征为例）

想象你在训练一个猫狗分类器，但里面罕见地出现了几张“无毛猫”的图片：

- **Focal Loss 怎么做**：当模型看到“无毛猫”时，因为没见过，预测概率 p 很低， $(1 - p)^{\gamma}$ 接近 1，Loss 很大。而看到普通猫狗时，预测极准， p 接近 1， $(1 - p)^{\gamma}$ 趋近 0。**结果**：整个模型的总 Loss 被“无毛猫”主导，强迫网络去关注它。
- **RMSprop 怎么做**：当反向传播把“无毛猫”的 Loss 变成梯度传回网络时，负责识别“无毛猫褶皱特征”的那几个特定神经元的权重 θ ，因为过去几千步里它的梯度都是 0（分母 $\sqrt{v}$ 极小），所以这几个权重的**更新步长瞬间飙升**。**结果**：负责这个特征的局部神经元瞬间完成了疯狂学习。

Adam已经很性感了，为什么还要AdamW (Weight Decay)

一、破案的关键：为什么 SGD 时代 L2 完美的等价于 Weight Decay?

在推导之前，我们先厘清这两个概念在定义上的本质区别：

- **L2 正则化 (L2 Regularization)**：是一种修改损失函数 (Loss) 的手段。在原始 Loss 后面加一个惩罚项，指望通过反向传播产生的梯度来约束权重。
- **权重衰减 (Weight Decay)**：是一种直接修改参数更新步骤的手段。在每次更新时，强行让权重乘以一个小于 1 的系数（统一按比例缩水），它本不需要和梯度扯上关系。

教案的第一步推导非常漂亮，它用代数证明了在 SGD 里它们为什么会合二为一：

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\mathbf{g}_t$$

- **物理直觉**：在 SGD 中，由于学习率 η 是一个标量（全局唯一的数字），所以无论是更新原始梯度 $\mathbf{g}_t$ ，还是执行 L2 产生的惩罚项 $\lambda\theta_t$ ，它们面对的都是同一个步长 η 。
- **结果**：参数 θ_t 完美的、均匀的、以 $(1 - \eta\lambda)$ 的比例向原点收缩。这被称为“径向收缩”（也就是所有维度的参数，都老老实实按统一比例变小）。

Adam 的预条件器

Adam 维护两个状态变量：

- 一阶矩 $\mathbf{m}_t$ ：梯度的指数移动平均（类似 Momentum）。
- 二阶矩 $\mathbf{v}_t$ ：梯度平方的指数移动平均（衡量梯度的大小）。

更新时，Adam 对每个参数坐标 i 执行：

$$\theta_{t+1}^{(i)} = \theta_t^{(i)} - \eta \cdot \frac{\hat{m}_t^{(i)}}{\sqrt{\hat{v}_t^{(i)} + \epsilon}}$$

其中 $\hat{m}$ 和 $\hat{v}$ 是偏差校正后的估计。分母 $\sqrt{\hat{v}_t^{(i)} + \epsilon}$ 是逐坐标不同的。

为了简洁，我们把这个逐坐标除法写成矩阵形式。定义对角矩阵 $\mathbf{D}_t$ ：

$$\mathbf{D}_t = \text{diag} \left(\frac{1}{\sqrt{\hat{v}_t^{(1)} + \epsilon}}, \frac{1}{\sqrt{\hat{v}_t^{(2)} + \epsilon}}, \dots \right)$$

那么 Adam 的更新就可以写成：

$$\theta_{t+1} = \theta_t - \eta \mathbf{D}_t \hat{\mathbf{m}}_t$$

如果硬把 L2 正则塞进 Adam

假设我们像在 SGD 中一样，把正则项加入损失函数，那么在计算梯度时就会多出 $\lambda\theta_t$ ：

$$\nabla \mathcal{L}_{\text{reg}} = \mathbf{g}_t + \lambda\theta_t$$

一阶矩 $\hat{\mathbf{m}}_t$ 就会变成 $\hat{\mathbf{m}}_t^{\text{reg}} = \hat{\mathbf{m}}_t + \lambda\theta_t$ （这里做了简化，忽略 EMA 的影响，但不影响本质）。

代入 Adam 的更新公式：

$$\begin{aligned}\theta_{t+1} &= \theta_t - \eta \mathbf{D}_t(\hat{\mathbf{m}}_t + \lambda\theta_t) \\ &= \theta_t - \eta \mathbf{D}_t \hat{\mathbf{m}}_t - \eta \lambda \mathbf{D}_t \theta_t\end{aligned}$$

请看最后一项！它是 $\eta \lambda \mathbf{D}_t \theta_t$ ，而不是我们想要的 $\eta \lambda \theta_t$ 。 $\mathbf{D}_t$ 这个对角矩阵插入了参数和收缩因子之间。

3. 用具体数字感受“失真”

假设只有两个参数 θ_1 和 θ_2 ，当前值都是 1.0， $\eta \lambda = 0.01$ （我们希望每次把它们都缩小 1%）。

正常的 Weight Decay：两个参数都变成 $1.0 - 0.01 \times 1.0 = 0.99$ 。公平、均匀。

Adam 加 L2 正则的“伪 Weight Decay”：

- 假设参数 1 的梯度一直很大，所以 $\hat{v}_1$ 很大，比如 $\hat{v}_1 = 100$ ，那么分母 $\sqrt{100} + \epsilon \approx 10$ ， $\mathbf{D}_1 \approx 1/10 = 0.1$ 。
- 参数 2 的梯度一直很小， $\hat{v}_2 = 1$ ，分母 $\sqrt{1} + \epsilon \approx 1$ ， $\mathbf{D}_2 \approx 1$ 。

现在执行“伪 Weight Decay”：

- 参数 1: $\Delta \theta_1 = -\eta \lambda \mathbf{D}_1 \theta_1 = -0.01 \times 0.1 \times 1.0 = -0.001$ 。只缩小了 0.1%！
- 参数 2: $\Delta \theta_2 = -\eta \lambda \mathbf{D}_2 \theta_2 = -0.01 \times 1 \times 1.0 = -0.01$ 。缩小了 1%，正常。

悲剧发生了：梯度大、变化活跃的参数（参数 1）反而被衰减得很少；梯度小、不活跃的参数（参数 2）却被衰减得更狠。这和正则化的初衷完全背道而驰——我们希望活跃参数也受到约束，而不是放任它们疯长。

$$\theta_{t+1}^{(i)} = \theta_t^{(i)} - \eta \cdot \frac{\hat{m}_t^{(i)}}{\sqrt{\hat{v}_t^{(i)} + \epsilon}}$$

梯度更新公式长这样

三、AdamW 的拨乱反正：把权力关进制度的笼子

洛什奇洛夫等人的做法简单粗暴且极具数学美感：既然 L2 放进梯度会被 $\mathbf{D}_t$ 扭曲，那我就把正则项彻底从梯度和分母里剥离出来！

看 AdamW 的最终合体公式：

$$\theta_{t+1} = (1 - \eta \lambda) \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t + \epsilon}}$$

1. 左边半大括号 $(1 - \eta \lambda) \theta_t$ ：恢复了类似 SGD 的纯正 Weight Decay。不论你当前的梯度有多大，所有维度的参数在每一步更新前，首先统一无条件按固定比例缩小。完美的径向收缩达成了。
2. 右边半大括号：纯净的 Adam 更新。里面的 $\hat{\mathbf{m}}_t$ 和 $\hat{\mathbf{v}}_t$ 只由真实的原始梯度 $\mathbf{g}_t$ 决定，再也不会被 L2 正则化的残留梯度污染。

我们本来指望L2劫富济贫，现在开始劫贫济富这不他妈闹吗

系统热力学

1. 随机梯度训练 = 参数在损失曲面上做“布朗运动”

回忆一下：真正的梯度下降（GD）每一步都沿着整个数据集的精确梯度方向走一小步。但现实中，我们用 mini-batch SGD，每一步的梯度只是在一个小批量数据上计算的，是对真实梯度的带噪估计。

可以把这个关系写成：

$$\mathbf{g}_{\text{SGD}} = \mathbf{g}_{\text{true}} + \epsilon$$

其中 ϵ 是梯度噪声。

这带来的直接影响是：参数不再是沿着一条光滑的曲线滑向谷底，而是像一个在水中被随机撞击的花粉颗粒，在损失曲面上做随机游走，一边被真正的梯度拉向谷底，一边被噪声推来推去。

学习率 η 的角色是什么？它不仅控制着每一步沿梯度方向走多远，同时也放大了噪声的幅度。学习率越大，随机游走每一步的步子就越大，参数轨迹就越“疯癫”，探索的范围就越广。

2. 损失曲面的地形：尖锐盆地 vs. 平坦盆地

现在来看损失曲面的“地形”。之前第一部分我们学了，极小值点（盆地底部）的曲率可以不同：

- **尖锐盆地**：谷底很窄很陡。参数稍微偏一点，损失就会急剧上升。
- **平坦盆地**：谷底宽阔平缓。参数在附近晃悠，损失不会涨太多。

哪个更好？通常是**平坦盆地**。因为测试数据和训练数据不完全相同，如果参数落在了尖锐盆地，测试时稍微偏移一点就会导致性能骤降。平坦盆地的解对参数扰动不敏感，泛化能力往往更强。

3. 学习率如何影响“你最终掉进哪个盆地”？

这是整节的核心直觉。

想象你从山上随机一个点开始往下滑，同时被随机噪声推搡。你最终会停在哪个山谷里？

- **如果一开始学习率很大**：你步子很大，噪声也大。你就像在一个高温度的环境中，有足够的动能翻越小山脊，越过狭窄的沟壑。你不会被困在遇到的第一个小坑里，而是可以继续探索，直到找到一个足够宽、足够深的盆地。
- **如果后来学习率越来越小**：你步子变小，噪声也变小。你就像在逐渐降温，动能慢慢耗散。一旦你进入了一个宽阔平坦的盆地，低温让你没有足够能量翻越盆地的边缘，你就稳定在这个区域了。
- **如果一直保持大学习率**：即使在盆地底部，你依然被噪声推得到处乱跑，永远无法精细收敛，甚至会跳出好的盆地。

4. 从物理到数学：为什么叫“退火”？

这整套逻辑在物理学里有一个经典的对应：**模拟退火算法**。

在冶金学里，退火是指把金属加热到高温，让原子获得足够动能自由移动，然后**非常缓慢地冷却**。在冷却过程中，原子会自然地排列成能量最低、最稳定的晶体结构（平坦盆地），而不是卡在混乱的高能态（尖锐盆地）。

Cosine Annealing 就是把这个思想搬到了深度学习里：

- **初始大学习率** = 高温，系统有强噪声，参数可以自由探索，不被局部陷阱困住。
- **余弦曲线逐渐降低学习率** = 按照一个精心设计的时间表降温，让系统从“粗粒度全局搜索”平滑过渡到“精细局部收敛”。
- **最终降到很低的 $\eta_{\min}$** = 低温冻结，参数稳稳落在当前盆地底部，精细调整。

所以，Cosine Annealing 不只是让 loss 曲线“看起来降得漂亮”，它本质上是一个时变噪声注入和衰减策略，用来引导优化轨迹偏向更好的极小值盆地。

作业

题目一：峡谷大逃亡——2D 病态曲面与轨迹可视化

要求

- 不使用真实数据集，直接构造 Rosenbrock 函数
$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2。$$
- 纯 NumPy 手写实现 SGD 与 Momentum 优化器，二者必须从同一个糟糕初值（如远离最优点的位置）出发，对该函数进行优化。
- 绘制损失等高线图，并叠加 SGD 和 Momentum 的参数更新轨迹。
- 在若干采样点显式计算 Hessian 矩阵，报告局部条件数的变化。

题目二：长尾梯度解剖学——CE 与 Focal Loss 的梯度预算重分配

要求

- 构造一个极端不平衡的二分类任务，前景与背景样本比例约为 1:100。
- 分别使用交叉熵损失（CE）与 Focal Loss 训练模型至少 1 个 epoch。
- 在模型最后一层注册反向传播拦截逻辑，分别统计前景样本与背景样本贡献的平均梯度范数。
- 记录总体指标、少数类指标以及两类样本的梯度占比。

题目三：优化器状态逆向工程——一阶矩、二阶矩与断点续训灾难

要求

- 构建包含 AdamW 优化器与 CosineAnnealingLR 调度器的优化引擎，并在其上训练模型。
- 训练 3 个 epoch 后，保存 `model.state_dict()`、`optimizer.state_dict()` 和 `scheduler.state_dict()`。
- 强行解析 `optimizer.state_dict()`，定位某一层参数对应的一阶矩与二阶矩张量。
- 可视化一阶矩、二阶矩的数值分布，并解释其反映的历史梯度统计含义。
- 故意只恢复模型权重，不恢复优化器与调度器状态，比较后续 10 个 step 的损失抖动与学习率轨迹偏差，并分析原因。

题目四：学习率调度与平坦极小值探针

要求

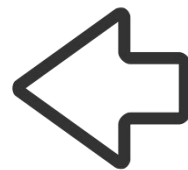
- 分别使用常数学习率与 Cosine Annealing 两种调度方案训练同一模型。
- 训练结束后，对最终模型参数施加多个尺度的随机扰动： $\theta' = \theta + \alpha \varepsilon$ 。
- 绘制扰动幅度 α 与损失上升量之间的关系曲线。
- 分析两种调度策略是否落入了不同尖锐程度的极小值邻域，并讨论平坦性与泛化能力的关系。

模块二：PyTorch与深度学习训练闭环

系统级防弹架构、依赖倒置与工业级 Trainer 逆向工程



主讲人：利群他爹



这是利群

那我们需要思考模型到底是什么

一、训练系统：从脚本到状态递推器

这是观念上的第一次升级。别再把写的 `train.py` 只看做一个“先 forward 再 backward 然后 step”的流程脚本了。

思考这个公式：

$$S_{t+1} = F(S_t, B_t, C)$$

- S_t ：时刻 t 的全量训练状态。这不是几个权重，而是一个包含了模型、优化器动量、学习率、随机数种子等一切会影响下一步走向的信息集合。
- B_t ：当前这批数据，包括所有随机增强后的结果。
- C ：静态配置，比如模型结构、设备类型等不变的东西。
- F ：一个确定的函数，输入当前状态和数据，输出下一个状态。

你要建立的认知：这个公式意味着，训练过程是**确定的**。只要你从完全相同的 S_t 出发，喂给它完全相同的数据 B_t ，它就能走到完全相同的 S_{t+1} 。

Resume 的本质：断点续训不是在“加载权重接着跑”，而是在尝试从某个历史时刻的状态点 S_t 重新启动，让系统继续在原来的动力学轨道上演化。只要 S_t 有任何一个部分被遗漏或改变，后续的轨迹就会永久性地偏离，进入一个“平行宇宙”。教案里让你“逼自己承认”的那三句话，就是这个意思。

依旧可复现性

- 可复现性不仅体现在完整项目的复现
- 当你的训练被挂起，回溯到上一checkpoint也需要可复现性

1. AMP Scaler (自动混合精度缩放器)

它的作用是动态调整梯度缩放因子，防止小梯度在低精度计算中下溢为0。

- **原理**：反向传播前，loss 乘以一个大数 s_t ；反向传播后，梯度再除以 s_t 。这个 s_t 本身是根据是否发生数值溢出而动态调整的，是一个随时间变化的状态。
- **如果漏存了它**：恢复训练时， s_t 会以默认值重新开始。这会导致训练初期，梯度的有效数值尺度发生突变，就像换了一个学习率，很可能导致 loss 剧烈震荡甚至直接 NaN。这不是误差，是数值制度的突变。

2. RNG State (随机数生成器状态)

这比 AMP 更隐蔽。训练中无处不在的随机性，都由随机数生成器控制。

- **它控制什么**：数据增强的随机参数、Dropout 层随机丢弃的神经元、某些 Sampler 的数据打乱顺序。
- **为什么不能只存种子**：一个典型的错误想法是：“我保存了随机种子，恢复时重新设置就行”。但 RNG 的状态不仅仅是初始种子，还包括它已经被消耗了多少次。恢复训练需要的是那个时刻的内部状态，这样才能保证下一个随机数产出和上次连续训练时完全一样。
- **如果漏存了它**：恢复后第一步，数据增强结果变了，Dropout mask 变了。你喂给模型的 B'_t 和原来的 B_t 不同，即使权重一样，梯度也会不同，轨迹就此分叉。这就是教案里说的，“一旦 RNG State_t 发生错位，系统更新就从...变成...”。

结论：想实现完美 Resume，你必须把所有随机数源（Python, NumPy, PyTorch CPU, PyTorch CUDA 每张卡）的内部状态都打包进你的快照。

这次是真callback

三、架构救赎：控制反转与 Callback 模式

这解决的是“代码怎么组织才不腐烂”的问题。

什么是“面条代码”？ 教案里那个把所有功能都塞进主循环的代码块就是。它的特点是：核心数学逻辑（训练）和非核心的工程逻辑（记录、保存、验证）纠缠不清。

它的“臭”在哪里？ 不是不优雅，而是耦合。想加一个新功能（比如想每100步发一封邮件），你就得修改训练循环这个核心文件，极易引入 bug。

解决方案：控制反转

- **Trainer 核心：**只保留一个极简的状态机，负责推进核心循环。它在关键节点（一个 epoch 开始、一个 step 结束、准备保存 checkpoint 时）发射一个信号（Event）。
- **Callback 回调：**所有外围功能都写成独立的插件，比如 `LoggerCallback`、`CheckpointCallback`。它们只订阅自己关心的事件并执行相应操作。

这带来的好处是决定性的：Trainer 不再依赖具体的日志或保存逻辑，你可以在完全不碰 Trainer 源码的情况下，通过增删回调来定制训练流程。这就是一个可扩展系统的根基。PyTorch Lightning 和 Hugging Face Trainer 强大，不是因为它们有多少功能，而是因为它们的架构支持功能的无限扩展。

四、序列化危机：告别 Pickle，拥抱 safetensors

这部分讲的是“用什么文件格式存”。

torch.save 的陷阱：它默认用 Python 的 `pickle` 协议。Pickle 是一个强大的序列化工具，但它把数据和代码指令混在一起。

- **安全风险：**反序列化一个 pickle 文件的过程，就像执行一个打包好的程序。一个恶意构造的 `.pth` 文件可以在你加载时，在你的电脑或服务器上执行任意命令。
- **I/O 瓶颈：**Pickle 是为 Python 对象设计的，不是为大规模张量优化的。加载一个大模型时，它需要重建大量 Python 对象，效率很低。

safetensors 的现代方案：

- **它是什么：**一种专门为张量设计的安全、快速的文件格式。
- **优势：**
 1. **安全：**它只包含数据和描述数据形状、类型的 header，**不包含任何可执行代码。**
 2. **快速：**扁平的布局使得它可以被零拷贝（通过内存映射）读取，加载速度极快，在多卡或大模型场景下优势巨大。
- **技术边界：**它只能存纯张量。所以，一个完整的 checkpoint 必须是一个组合包：
 - **张量载荷：**用 `safetensors` 存模型和优化器的张量。
 - **元数据包：**用 JSON 或 YAML 存 `epoch`、`global_step`、`config`、RNG 状态等非张量信息。

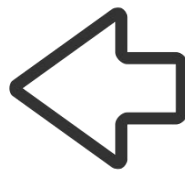
你要建立的认知：`safetensors` 不是另一个 `torch.save`，它是一种全新的存储哲学，强制你把数据和代码、张量和元信息分开。

模块二：PyTorch与深度学习训练闭环

混合精度、数值病理学与性能 Profiling



主讲人：利群他爹



这是利群

OOM不是玄学，是显存会计学

- 会计恒等式：资产+费用=负债+所有者权益+收入
- 总显存 $\approx$ 模型参数 + 参数梯度 + 优化器状态 + 中间激活 + 杂项
- 看到 CUDA out of memory，你的第一反应不能是“那就把batch size调小”，而是“让我看看账本，到底是谁在吃显存？”。

- 模型参数 M_{param} ：这就是你模型本身的“体重”。
- 参数梯度 M_{grad} ：每次反向传播算出的“更新方向”，体量和模型参数一样大。
- 优化器状态 M_{opt} ：优化器自己存的“私房钱”。
 - SGD：几乎不存，最省。
 - Momentum：多存一份“速度”记录，参数量翻倍。
 - Adam/AdamW：多存两份，“一阶矩”和“二阶矩”，参数量是SGD的三倍。这是显存大户。
- 中间激活 M_{act} ：真正的罪魁祸首。为了在反向传播时计算梯度，前向传播中每一层的计算结果（激活值）都得被记下来。这个“账本”的大小和你的batch size成正比。
- 杂项 M_{misc} ：CUDA内核需要的临时空间、显存碎片等。

1. 为什么 batch size 对显存的杀伤远大于很多超参？因为它线性放大多层激活缓存。
2. 为什么推理往往能跑更大 batch，而训练不行？因为推理不需要为反向传播保留整条计算图的激活。

FP16 vs BF16：一场“范围”与“精度” 战争

混合精度训练（AMP）能提速和省显存的关键，在于使用更少位宽的数据格式。但“位宽减半”带来的不仅仅是存储变小，更是数值特性的剧变。

IEEE 754 标准定义了浮点数的结构：1 位符号位（Sign）+ 指数位（Exponent）+ 尾数位/小数位（Mantissa/Fraction）。

格式	总位数	符号位	指数位	尾数位	动态范围 (Dynamic Range)	精度 (十进制有效数字)
FP32	32	1	8	23	$\sim 10^{-38}$ 到 10^{38}	~ 7 位
FP16	16	1	5	10	$\sim 5.96 \times 10^{-8}$ 到 65504	$\sim 3 - 4$ 位
BF16	16	1	8	7	$\sim 10^{-38}$ 到 10^{38}	$\sim 2 - 3$ 位

- 1. OOM 有时不是“理论总量超了”，而是 CUDA allocator 碎片化导致大块连续内存申请失败。
- 2. 显存优化从来不是单点技巧，而是参数、激活、优化器、checkpointing、精度格式协同设计。
- 3. 混合精度真正省下来的，往往首先是激活和部分算子中间张量，而不是所有状态都自动减半。

- 1. 为什么 batch size 对显存的杀伤远大于很多超参？因为它线性放大多层激活缓存。
- 2. 为什么推理往往能跑更大 batch，而训练不行？因为推理不需要为反向传播保留整条计算图的激活。

AMP与GradScaler：精度调度的艺术与补丁

1. FP16 的阿喀琉斯之踵：溢出 (Overflow / Underflow)

FP16 虽然有 10 位尾数（精度不错），但它的指数位只有 5 位。

- **上溢出 (Overflow):** 最大只能表示 65504。如果梯度更新出现稍微大一点的数值，就会变成 `Inf`，导致 NaN 传播。
- **下溢出 (Underflow):** 最小的正非零正常数是 6.10×10^{-5} 。深度学习反向传播末期的梯度往往非常小（比如 10^{-6} ），在 FP32 下毫无问题，但在 FP16 下会直接被截断为 0（梯度湮灭）。

这就是为什么 FP16 往往面临的问题不是“算得不够准”，而是“算出来直接是 0 或 Inf”。

2. BF16 的崛起：牺牲精度换取范围

Google/DeepMind 发明的 BF16 (Brain Float 16) 核心思路是：深度学习模型对范围 (Range) 的敏感度远高于对精度 (Precision) 的敏感度。BF16 直接砍掉了 FP32 的 16 位尾数，保留了完整的 8 位指数。

- **优点：** 动态范围和 FP32 完全一样，永远不用担心下溢或上溢。不需要复杂的 `GradScaler`。
- **缺点：** 只有 7 位尾数，精度较低，导致步长非常不平滑。对于某些对数值敏感的任务（如极度依赖精密梯度的生成模型），BF16 可能会掉点。

GradScaler 的工程救赎

三、GradScaler 的工程救赎

既然 FP16 容易发生梯度下溢（变成 0），为什么我们还要用它？因为在 BF16 普及前（Nvidia 直到 Ampere A100 才支持硬件级 BF16），FP16 是唯一的选择。

`GradScaler`（梯度缩放器）就是为了拯救 FP16 的下溢出而诞生的工程补丁。

核心原理：

1. **放大 Loss**：在反向传播开始前，给 loss 乘上一个很大的缩放因子 s （例如 $s = 65536$ ）： $\tilde{\mathcal{L}} = s \cdot \mathcal{L}$
2. **安全反向传播**：根据链式法则，所有的梯度都会被放大 s 倍。原本 10^{-6} 的梯度变成了 0.065，安全进入 FP16 的表达式区间，没有变成 0。
3. **更新前缩回**：在 `optimizer.step()` 更新权重之前，将梯度除以 s 缩放回来，保证权重更新的步长是正确的（这一步通常会被强转回 FP32 计算）。

动态调整机制：如果 s 太大，会导致上溢变成 `Inf`。PyTorch 的 `GradScaler` 会动态监控：

- 如果连续 N 步没有出现 `Inf`，说明很安全，尝试把 s 翻倍（探顶）。
- 如果突然遇到 `Inf` 或 `NaN`，跳过当前 batch 的更新（丢弃梯度），并将 s 减半（退避）。

关键认知：`GradScaler` 不会改变学习率，也不会提升算术的绝对精度。它完全是一个对抗浮点数格式底层的数值工程 Hack。

梯度累加 (Gradient Accumulation)

四、梯度累加 (Gradient Accumulation)

当你把模型做大，或者输入变长，即便用了 AMP 还是 OOM 怎么办？这就需要梯度累加。

如果你想用 Batch Size = 32，但显存只允许跑 Batch Size = 8：

1. 把 Batch 32 切成 4 个 Micro-batch（每个大小为 8）。
2. 循环 4 次：跑前向、算 loss、算反向传播。**关键点：不清空梯度**，而是让 4 次算出来的梯度在叶子节点处自动累加（这就是 PyTorch 中梯度的默认行为：`+=`）。
3. 循环 4 次后，此时显存里积累的梯度总和，等效于 跑了一次真实的大 Batch Size。
4. 调用 `optimizer.step()` 更新权重，然后 `optimizer.zero_grad()` 清空。

有效批大小 (Effective Batch Size)：

$$\text{Effective Batch Size} = \text{Micro Batch Size} \times \text{Accumulation Steps} \times \text{Number of GPUs}$$

注意：

- 梯度累加在数学上几乎完全等效于直接使用大 Batch Size（前提是不涉及带有跨 batch 统计量的层，如 BatchNorm）。
- 在使用平均 loss 时，需要将每个 micro-batch 的 loss 除以 `accumulation_steps`，保证最终累加梯度的缩放比例正确。

性能 Profiling：不要只看“跑完要多久”

五、性能 Profiling：不要只看“跑完要多久”

评估模型性能，必须建立更细粒度的指标体系。总时长受到 IO、系统抖动影响很大，我们要看的是核心吞吐能力。

1. **Step Time (ms/step)**: 每完成一次权重更新（一个 effective batch）所需的时间。
2. **Throughput (samples/s 或 tokens/s)**: 吞吐量。

$$\text{Throughput} = \frac{\text{Effective Batch Size}}{\text{Step Time (s)}}$$

这是衡量 GPU 压榨效率最核心的指标。

3. **Peak Memory (MB/GB)**: 峰值显存。用来判断是否还有压榨显存（增大 batch size）的空间。

更专业的 Profiling（如使用 PyTorch Profiler 或 NVIDIA Nsight Systems, `nsys`）会进一步拆解：

- 数据加载时间 (DataLoader bottleneck)
- CPU到GPU的内存拷贝时间 (Host to Device)
- CUDA Kernel 实际执行时间 vs GPU 空闲等待时间

